

Application of Computer Vision Models for Plant Counting in Rice Crops

Prof. Alejandro Manuel Lloveras

Specialization Program in Artificial Intelligence

Advisor: Eng. Juan Ignacio Cavalieri (FIUBA)

Co-advisor: Dr. Marcel Bentancor (UdelaR)

Examining Committee:

Dr. Eng. Facundo Luciana (UNT/FIUBA)

Spec. Eng. Alfonso Rafel (UNR/FIUBA)

Eng. Martín Horn (UNLu/FIUBA)

Translator's Note

Please be advised that this English text is a machine-assisted translation of the original thesis, which was written and formally approved in Spanish. This version was created for dissemination purposes, aiming to make the research findings accessible to a global audience. In line with this, all code in the accompanying repository is programmed and commented in English.

While efforts have been made to ensure fidelity to the source, potential discrepancies in nuance and terminology may exist. The original Spanish document should be considered the definitive and authoritative version. Text embedded within figures and tables remains in the original language.

Abstract

This report describes the development of a computer vision pipeline, developed in collaboration with Adecoagro. The objective is to automate plant counting in rice crops to improve efficiency and reduce monitoring costs.

The work involved knowledge of data analysis, statistics and spatial analysis, file management, image processing, vegetation indices, data augmentation, supervised and unsupervised machine vision, deep learning (CNN architectures), transfer learning, and hyperparameter optimization.

Acknowledgements

First, I would like to thank the advisors, who have accompanied me in this arduous task. I would also like to thank the examining committee for their interest in my thesis and the time dedicated to its correction.

Furthermore, I would like to thank the reviewers from the Faculty of Engineering of UBA, whose exhaustive style correction of each chapter and valuable advice were of great use during the writing of this report.

Last but not least, I want to thank my parents, who have always promoted curiosity and the desire for knowledge.

Contents

Abstract	iii
1 General Introduction	1
1.1 Introduction to crop sampling	1
1.2 Motivation	1
1.3 State of the art	2
1.4 Objectives and scope	3
2 Specific Introduction	5
2.1 Unmanned Aerial Vehicles (UAV)	5
2.1.1 Characteristics of the Drones Used	5
2.2 Data Processing	5
2.2.1 Image Labeling	6
2.2.2 Data Augmentation Technique	6
2.3 Artificial Intelligence Models Used	6
2.3.1 Convolutional Architectures (CNN)	7
2.3.2 YOLO Architecture	7
2.3.3 Fine-Tuning Technique (Transfer Learning)	8
2.4 DBSCAN Algorithm	8
2.5 Vegetation Indices: <i>Excess Green</i>	9
2.6 Principal Component Analysis (PCA)	9
2.7 Adaptive Histogram Equalization (CLAHE)	10
3 Design and Implementation	11
3.1 System Architecture	11
3.2 Image Capture	11
3.2.1 Crop Characteristics	12
3.2.2 Drone Flight Plan	12
3.3 Dataset Construction	14
3.3.1 Classes and Labeling Strategy	15
File Name Management	15
3.3.2 Data Augmentation	15
1-Stage <i>Data Augmentation</i> Strategy	16
Application for the False-Color Dataset	16
2-Stage <i>Data Augmentation</i> Strategy	16
Gentle <i>Data Augmentation</i> Strategy	17
3.3.3 Application of Vegetation Indices (False-Color Dataset)	17
<i>Excess Green</i> (ExG)	17
Dimensionality Reduction (2-PCA)	18
False Color (FC)	18
Color Burn	19
3.3.4 Soil Images	20
3.4 Image and Label Preprocessing	20

3.4.1	Tiling	21
	<i>Offset</i> (optional)	21
	Image Division	21
	Label Division (for training only)	22
	File Name Management	23
3.5	Adaptive Equalization (CLAHE)	23
3.6	Normalization (Range Scaling)	23
3.7	False Color Preprocessing (Optional)	23
3.8	Object Detection Model	24
3.8.1	Data <i>Split</i>	24
3.8.2	Model Training	24
3.8.3	Model Validation	25
3.8.4	Inference: Plant Detection Module	26
3.9	Prediction Post-processing	27
3.9.1	Label Reconstruction and Cleaning	27
3.9.2	Crop Line Detection	27
3.9.3	Weed Discrimination	29
3.9.4	Plant Counting and Density Calculation	29
3.10	Batch Processing	30
3.11	Metric Suite	31
4	Essays and results	33
4.1	Model tests	33
4.1.1	Tests with different architectures	34
4.2	Dataset Construction	34
4.2.1	Results with angled images	35
4.2.2	Results for different flight heights	35
4.3	YOLOv8 Configuration	36
4.3.1	<i>Transfer learning</i>	36
4.3.2	Hyperparameter tuning	37
	Weighted loss function (<i>Loss function</i>)	37
	Multi-scale training (<i>multi_scale</i>)	37
	SGD Optimizer	37
	<i>Momentum</i>	37
	L2 Regularization (<i>weight_decay</i>)	38
	<i>Dropout</i>	38
	Other hyperparameters explored	38
4.4	Performance of the best models	39
4.5	Model validation	40
4.5.1	Impact of IoU and confidence	41
4.6	Model testing	41
4.6.1	Impact of plant development on detection	42
4.7	Performance in production	44
4.7.1	Characterization of the productive system	45
	Critique of the traditional method	45
4.7.2	Impact of plant development on <i>performance</i>	46
4.7.3	Reflection on confidence and IoU threshold	46
4.7.4	Problematic cases	46
4.7.5	Impact of flight height on <i>performance</i>	47
4.8	Estimation of necessary resources	47
4.8.1	Contribution to productivity	48

5	Conclusions	51
5.1	General conclusions	51
5.2	Next steps	52
A	Inference results	55
B	Number of samples and measurement precision	57
B.1	Cluster sampling	57
B.2	Impact of sample size on <i>Accuracy</i>	62
C	Drone flight protocol	65
D	Hardware resources	67
	Bibliography	69

List of Figures

2.1	Example of data augmentation ¹ .	6
2.2	Characteristic structure of a CNN network.	7
2.3	Comparison between DBSCAN and <i>K-means</i> algorithms ² .	8
2.4	Example of the process of searching for neighboring points and detecting <i>outliers</i> ³ .	9
3.1	Inference pipeline diagram.	11
3.2	Error in plant count as a function of height.	13
3.3	Count error by plot as a function of height.	13
3.4	Diagram of <i>scripts</i> for dataset development.	14
3.5	Result of the <i>Excess Green</i> transformation.	17
3.6	Average histogram per channel for the dataset.	18
3.7	Result of dimensionality reduction with PCA.	19
3.8	Result of <i>Color Burn</i> blending.	19
3.9	Soil image processing.	20
3.10	Original image and image modified with <i>offset</i> .	21
3.11	Process of generating false color images.	24
3.12	Diagram of <i>scripts</i> for experimentation.	25
3.13	Diagram of <i>scripts</i> for validation and testing.	26
3.14	DBSCAN <i>cluster</i> assignment.	28
3.15	Crop line detection.	29
3.16	Weed detection (<i>outliers</i>).	30
4.1	Label correlogram.	35
4.2	F1-score curves as a function of confidence.	39
4.3	Confusion matrix for the validation set.	40
4.4	<i>Precision</i> curves as a function of confidence and <i>recall</i> .	41
4.5	Confusion matrix for the testing set.	42
4.6	Confusion matrix for advanced development plant.	42
4.7	Example of inference for a waterlogged lot.	44
4.8	Impact of IoU and <i>Confidence</i> on inference.	46

List of Tables

3.1	Field Data	12
3.2	First image capture (2/10/2024).	12
3.3	Second image capture (12/12/2024).	14
3.4	Roboflow Transformations.	16
3.5	Albumentations Transformations (YOLO <i>default</i>).	17
3.6	Values of the area covered by the photograph.	30
4.1	Distribution of labels by class.	35
4.2	Results for <i>Transfer learning</i>	37
4.3	Model training results.	39
4.4	Model validation results.	40
4.5	Model testing results.	42
4.6	RGB model response based on development and waterlogged terrain.	43
4.7	FC model response based on development.	43
4.8	Estimation of time for the counting task.	48
A.1	Computation time for batch processing.	55
A.2	Detections for the same sector at different heights.	56
D.1	Specifications of the NVIDIA Tesla T4 GPU.	67

***Dedicated to my family,
for their unconditional support.***

Chapter 1

General Introduction

This chapter addresses the problem of plant counting in rice crops, the motivation to make this task more efficient, the solutions currently available on the market, and the objectives and scope of the present work.

1.1 Introduction to crop sampling

Plant counting in crops is an essential practice in agriculture, providing critical information about planting density, crop health, and expected yield. Traditionally, this process has been carried out using manual sampling methods, in which the farmer takes samples from each lot, counts the plants present in representative plots, and then estimates the total quantity using statistical calculations. These methods present great variability in their results, depending on the criteria used by the specialist when selecting the plots, so the precision will depend on the bias introduced with this decision. Their experience and expertise are fundamental to ensure a representative measurement. On the other hand, counting is a demanding process that requires a considerable investment of time and can require full days of work, given that it is necessary to travel the entire extension of the lot, stopping on multiple occasions to take samples. Added to this is the time necessary for travel to the field and the impact of exogenous variables, such as weather conditions and the passability of the terrain, as well as access roads, which can considerably prolong the time required for this task.

1.2 Motivation

Adecoagro planted approximately 65,000 ha of rice in the 2024/25 campaign [1], distributed across four farms in Argentina (provinces of Santa Fe, Corrientes, and Entre Ríos) and two in Uruguay (in the city of Melo, Cerro Largo department) [2], and annually increases the planted hectares by an average of 5%. For the company, the rice business has represented around 15%. Therefore, it is crucial for them to be able to optimally monitor these crops.

To manage such a large number of hectares, Adecoagro has a large network of professionals distributed throughout the country, empowered by the use of drones, satellite images, meteorological data, and crop traceability, which can be consulted through their platform. Each lot manager has two assistants and is assigned 3 to 4 thousand hectares, equivalent to around 20 lots (each extending from 150 to 300 ha). Performing a plant count with traditional methods can take an estimated 2 hours per hectare, depending on the size of the lot, weather and terrain conditions, and the number of samples taken, being able to move within

the lot by walking or by motorcycle. That is, each lot requires between 300 to 600 man-hours of trained personnel. It can be seen that this process is hardly scalable and has a great economic impact for the company. On the contrary, by performing sampling with drones, the time required can be considerably reduced, since the need to enter the lot is eliminated, because the operator can load the predefined flight plan and perform it autonomously from their vehicle. In this way, all the necessary images to measure an entire lot can be captured in less than 30 min. Furthermore, it is easy to scale the number of samples captured if necessary. With this solution, the entire counting process is automated, time is considerably reduced, mobility within the field is facilitated and streamlined, the necessary training to perform the task is minimized, and the bias introduced by the operator is eliminated. Including the image loading and processing time by the AI pipeline, plant counting could be completed in 1 hour using the developed solution. This represents a potential 8-fold increase in productivity. By improving efficiency, reducing resource use, and obtaining more precise data at a lower cost, the number of annual counts per lot could be increased, as this solution allows for regular monitoring of crops and more efficient and sustainable management.

1.3 State of the art

Currently, there are various solutions for automatic plant counting using computer vision techniques in different types of extensive crops, such as corn, wheat, or soybeans. However, a similar service for rice crops is not available on the market. According to the research carried out within the framework of the present study, no international providers offering a solution to this problem have been identified. Currently, analysis services for rice cultivation include:

1. Plant stress report: to visualize unhealthy areas of the field and address problems at an early stage, avoiding significant losses.
2. Average plant density report: to identify areas with below-average plant density and apply fertilizers precisely.
3. Weed detection report: to detect affected areas and perform variable herbicide application.
4. Yield report: based on panicle volume measurement, it allows estimating the potential crop yield.

In the academic field, numerous investigations stand out that have applied artificial vision models (*Computer Vision*, CV) in automatic plant counting for various crops, with the aim of comparing the number of germinated plants with the quantity of seeds planted and thus determining the effective crop density. Traditionally, different artificial vision techniques have been applied in agriculture, especially the use of color spaces. Given the nature of the problem, in which green color is predominant in plant images, the use of vegetation indices based on color spaces allows for easy discrimination between plants and background. García-Martínez et al. applied normalized cross-correlation (NCC) along with CIELAB color space to count corn plants [3]. Valente et al. used convolutional neural networks along with vegetation indices (*Excess Green*) and Otsu to identify and count spinach plants [4]. It is common to work with high-resolution images captured by unmanned aerial vehicles (*Unmanned Aerial Vehicle*, UAV), popularly known as drones, as they allow covering large areas of cultivation in a practical and

economical way. Over the years, different CNN architectures have been implemented for plant detection and counting. Madec et al. explored the use of Faster R-CNN and TasselNet for counting wheat spikes [5]. Xu et al. applied Mask R-CNN along with YOLOv5 to count leaves in corn seedlings [6]. Also, Ammar et al. compared the performance of Faster R-CNN against EfficientDet for palm counting [7]. Wang et al. used RetinaNet for the detection and counting of corn ears [8]. In addition, Karami et al. proposed the use of CenterNet for automatic corn counting with low data volume (*few-shot learning*) [9]. However, the use of drone images presents an additional challenge, as the objects to be detected are commonly small, which makes the use of standard models difficult. To solve this, Wang et al. incorporated the Kalman filter into the YOLOv3 model [10]. For their part, Luo et al. combined K-means++ and Soft-NMS to YOLOv3, with the same goal [11]. Furthermore, in many cases, convolution networks are used combined with additional stages that complement their operation by facilitating feature extraction and allowing model learning. Tatini et al. proposed adding an additional SUFF-type layer to YOLOv4, which allows extracting better feature maps for rice fields [12]. On the other hand, Yang et al. combined GhostNet with YOLOv5 and applied *Efficient Intersection over Union* (EIoU) to optimize performance [13]. In contrast, Luo et al. replaced the YOLOv5 *backbone* with ASRes2Net and included *Efficient Channel Attention* (IECA) to improve classification accuracy [14].

Recent studies have attempted to address the problem of plant counting for the specific case of rice crops. In this regard, Yeh et al. proposed the use of the YOLOv4 model replacing the ReLU activation function with Mish with excellent results [15]. On the other hand, Yao et al. propose the use of P2PNet-EFF with an efficient multi-scale attention (EMA) *backbone* to achieve accurate plant counting [16]. Although these investigations have contributed to expanding the field of knowledge, with good precision and *recall* metrics, it is essential to delve deeper into this domain to be able to offer practical and reliable solutions. It is crucial that the solutions provided adapt to the particularities of local crops and the challenging characteristics presented by intensive agriculture, which has a considerably higher plant density than academic studies generally carried out in small trial plots.

1.4 Objectives and scope

The objective of the present work is to automate plant counting in rice crops to reduce the use of human resources, compared to those required by traditional sampling methods, which are time-intensive and require the presence of specialized personnel in the field. By automating this process, it is expected to streamline the addressing of this need and increase the number of counts per harvest, which will allow for regular monitoring of crops efficiently and at a lower cost. The work includes:

- Field capture of high-resolution aerial images.
- Image processing and classification.
- Image labeling for dataset creation.
- Evaluation of different artificial vision architectures.
- Development of the AI *pipeline*:

- Plant-object detection in images.
 - Tracing of crop distribution line.
 - Discrimination of predictions between plants and weeds.
 - Counting the total number of plants per lot.
 - Estimation of the area covered by the photograph.
 - Calculation of density per hectare.
- Optimization of the result through metric analysis.

Chapter 2

Specific Introduction

This chapter presents the techniques and tools used to develop the present work.

2.1 Unmanned Aerial Vehicles (UAV)

Unmanned Aerial Vehicle (UAV) technology represents a revolution in multiple industries, including an increasingly crucial role in agriculture. These devices are equipped with a wide variety of sensors, cameras, and navigation systems (such as GPS), which allow them to fly autonomously (following a predefined flight plan) or be remotely controlled via radio. In agriculture, drones offer an invaluable aerial perspective for crop monitoring, as they enable early detection of diseases, pests, or water stress through high-resolution photographs and videos, multispectral or thermal images. They can capture a large amount of data that facilitates precise decision-making regarding the application of fertilizers, pesticides, or irrigation. This leads to resource optimization and improved yields. Additionally, there are large-span drones equipped for precision planting and selective spraying.

2.1.1 Characteristics of the Drones Used

For image capture, DJI Mavic 2 Pro drones were used. This model has an estimated autonomy of 30 minutes under ideal conditions (no wind, at a constant speed of 25 km/h) and reaches a maximum speed of 72 km/h (~ 20 m/s). These specifications allow it to fully cover a 200 ha plot and capture sample photographs without requiring recharging or battery changes. The drone has a dual GPS + GLONASS geolocation system that improves flight stability and location accuracy. The altitude accuracy is ± 0.5 m flying with GPS only and ± 0.1 m when visual sensors are also used. The DJI Mavic 2 Pro is equipped with a Hasselblad L1D-20c camera with a 1-inch CMOS sensor and 20 MP resolution. The lens has a 28 mm equivalent focal length, an adjustable aperture from $f/2.8$ to $f/11$, and focuses from 1 m to infinity. Photographs can be saved in JPEG or DNG (RAW) format, with a maximum resolution of 5472×3648 px. The camera stabilization system is based on a 3-axis mechanical *gimbal* with $\pm 0.01^\circ$ precision, which allows obtaining sharp, undistorted images during flight.

2.2 Data Processing

For the present work, an original dataset was created from images provided by the client. In addition, data augmentation techniques were applied to enrich the

dataset and improve model generalization.

2.2.1 Image Labeling

For image labeling, the Roboflow tool was used, with the YOLOv8 PyTorch TXT label standard, a modification of the YOLO Darknet format. According to this format, the dataset is organized in a folder structure (`train`, `valid`, `test`) and subfolders (`images`, `labels`), indicated within the `data.yaml` file, where the number of classes and their names are also detailed. For each image in the dataset, there is a `.TXT` file of the same name, located in the corresponding folder (e.g., `../train/labels`).

Each line of the file represents a label of the form:

```
class_id center_x center_y width height
```

The label details the class and normalized coordinates (from 0 to 1) with the relative position of the centroid (`center_x`, `center_y`), the width (`width`), and the height (`height`) of the *bounding box*.

2.2.2 Data Augmentation Technique

Data augmentation is a set of techniques that allow generating new images to enrich the dataset, which improves generalization. Unlike "synthetic data," which are completely new, "augmented data" are created by applying transformations to the original set. Particularly in the domain of computer vision, these techniques can simulate various scenarios that the model might encounter in the real world and help it perform better in different conditions, regardless of the orientation, size, or lighting conditions of the photograph. Examples of the most common transformations can be seen in figure 2.1.



FIGURE 2.1. Example of data augmentation ¹.

2.3 Artificial Intelligence Models Used

The utilization of *Deep Learning* (DL) models has demonstrated great utility in agriculture by responding better to the variability of input data (lighting and climatic conditions, terrain and crop characteristics, local weeds, plant growth stage, etc.). This also allows the application of *data augmentation* and *transfer learning*.

¹ Adaptation of the image from the *paper* <https://www.mdpi.com/2077-0472/14/3/331>

2.3.1 Convolutional Architectures (CNN)

Convolutional Neural Networks (CNN) are the basis of artificial vision models for classification, detection, and segmentation of objects. They operate in two stages: feature extraction and classification.

Their strength lies in autonomously extracting features (*feature extraction*) through convolutional layers, which apply filters (*kernels*) pixel by pixel to identify patterns and create feature maps. Subsequent *down-sampling* layers reduce dimensionality (*pooling*) and abstract the main characteristics. The concatenation of these structures allows identifying complex patterns.

Finally, as can be seen in figure 2.2, a flattening layer (*flatten layer*) transforms the multidimensional tensor into a unidimensional vector, which feeds the *fully-connected* layers of the classification stage, which function as a classic neural network.

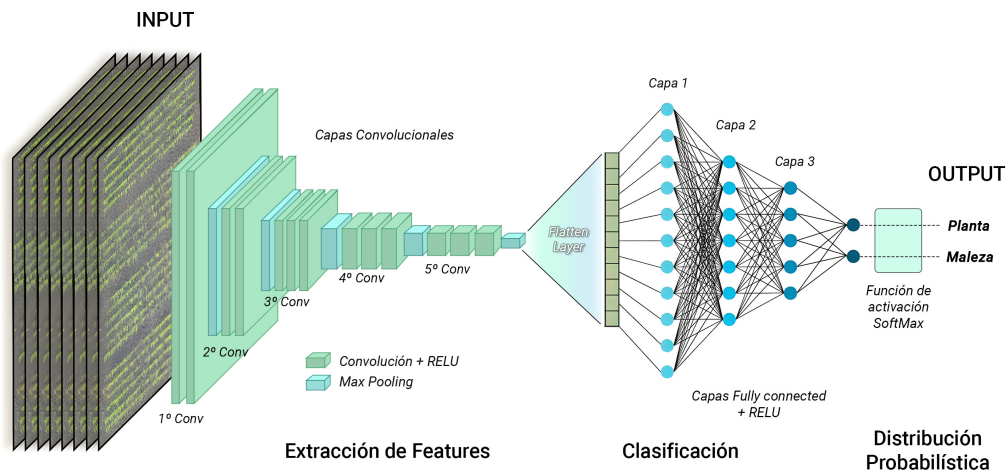


FIGURE 2.2. Characteristic structure of a CNN network.

2.3.2 YOLO Architecture

The computer vision model YOLO, presented by Redmon et al. in the article «*You Only Look Once: Unified, Real-Time Object Detection*» in 2015, revolutionized artificial vision by allowing real-time object detection. Unlike previous architectures (R-CNN, Fast R-CNN, etc.) that used a two-stage process (region proposals and classification), YOLO unified everything in a single step, drastically reducing inference time. YOLO also offers translation and scaling invariance from v3 with *Feature Pyramid* and rotation invariance when applying *data augmentation* (especially in v8). Successive versions have improved precision, performance, and training and inference speed. In the context of plant detection in productive environments, YOLOv8's "*anchor-free*" feature is crucial since it simplifies training and makes it robust to variations in object shape and scale. This allows it to recognize plants visually different from those in the training set (due to phenological stage, plant density, or leaf position).

2.3.3 Fine-Tuning Technique (Transfer Learning)

Transfer Learning is a set of techniques that allows transferring the learning of a model to other specific domains. The *Fine-Tuning* technique, in particular, allows loading the weights of pre-trained models into each neuron of the network and using them as a starting point for a new training. This facilitates some layers learning the characteristics of the dataset, while others remain immutable (a process known as *freeze* or “parameter freezing”). The main advantages of this technique are: optimization, as it reduces computational effort by using pre-trained models; cognitive transfer, allowing models trained for one task to function well in other domains; adaptability, facilitating customization for specific contexts; and reduction of data requirements, achieving high precision with smaller datasets.

2.4 DBSCAN Algorithm

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) is a robust and efficient unsupervised learning *clustering* algorithm. As seen in figure 2.3, it largely outperforms alternatives like *K-means* by not requiring prior knowledge of the number of *clusters*. Its ability to identify data groupings and discriminate noise (isolated points) makes it ideal for data with high variability, such as aerial crop photographs.

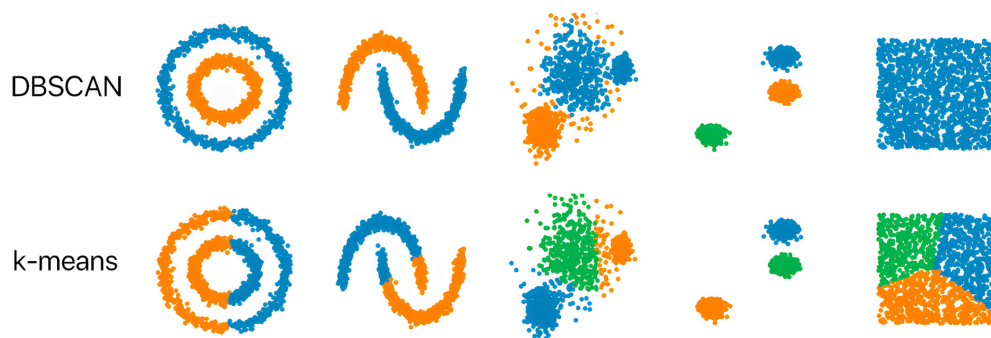


FIGURE 2.3. Comparison between DBSCAN and *K-means* algorithms ².

For its implementation, DBSCAN requires two key parameters: epsilon (ϵ), maximum distance for two points to be considered neighbors, and minimum points (`minPoint`), threshold of points necessary to form a *cluster*. The algorithm arbitrarily selects a point and verifies how many neighbors exist within the radius ϵ . If it exceeds `minPoint`, those points are labeled as part of the same *cluster*. This process expands iteratively, new circles are drawn from the detected points to find new neighbors within ϵ and progressively cover the entire *cluster*. Those points that do not meet the distance and quantity conditions are classified as “noise” (assigned the value -1).

²Image taken from the repository <https://github.com/NSHipster/DBSCAN/>

³Image taken from the KDnuggets article <https://www.kdnuggets.com/2020/04/dbscan-clustering-algorithm-machine-learning.html>

The search for nearest neighbors can be done with brute force (for sparse data), *ball tree* or *k-d tree* (with dense data).

2.5 Vegetation Indices: *Excess Green*

Vegetation indices are crucial tools in modern agriculture for non-destructive monitoring of crop health and coverage. They are based on mathematical combinations of spectral bands captured by remote sensors, such as satellites or drones. By analyzing reflectance in color spaces like RGB, HSV, or Lab, these indices highlight the biophysical properties of plants, allowing for the detection of stress, estimation of yields, and differentiation of crops from weeds, thus optimizing agricultural management. A particularly useful index is Excess Green (ExG), a color space algorithm applied to RGB images that emphasizes pixels with high information in the green channel. This is invaluable in agriculture for measuring crop coverage and evaluating plant health, differentiating organic material (plants, weeds) from other elements in the image. It is calculated as $ExG = 2G - R - B$, where G, R, and B represent the values of the Green, Red, and Blue bands, respectively. It is fundamental to apply Min-Max scaling to each RGB channel before calculation to homogenize their ranges and reduce the influence of variable illumination, thus ensuring a more robust and consistent index between different images. The result is a grayscale image where higher values correspond to vegetation. Commonly, a threshold is applied to segment the vegetation, creating a binary mask (white for plants, black for the background) that functions as a filter to crop plants from the original image. The choice of threshold varies according to lighting conditions and image characteristics.

2.6 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a statistical technique used for dimensionality reduction, especially in datasets with many interrelated variables, such as multispectral images. It seeks to simplify the information by transforming the correlated bands into a new set of uncorrelated principal components. The first

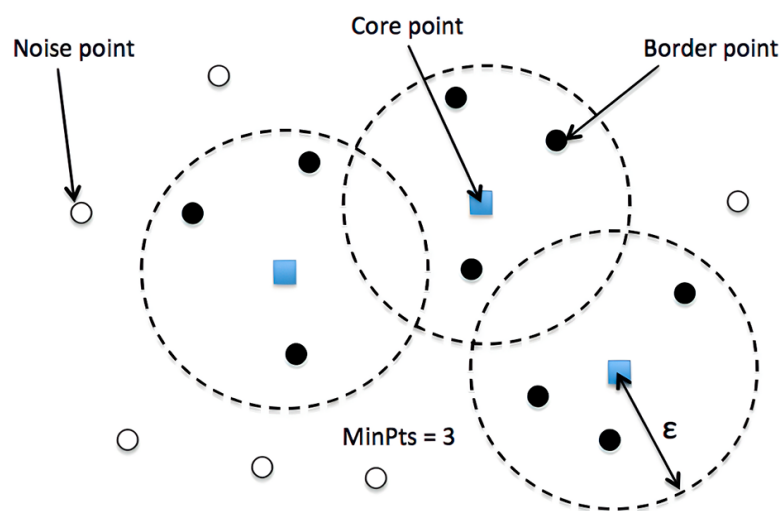


FIGURE 2.4. Example of the process of searching for neighboring points and detecting outliers³.

principal component captures the most variance (information), while subsequent ones capture the remaining variance, being all orthogonal to each other. This technique allows feature extraction (*feature extraction*) without significant loss of information, which makes the principal components ideal as new features for classification or segmentation algorithms. In addition, PCA contributes to noise reduction in images, as the components with lower variance usually contain noise, allowing a cleaner representation by discarding them.

2.7 Adaptive Histogram Equalization (CLAHE)

Contrast Limited Adaptive Histogram Equalization (CLAHE) is an advanced technique for enhancing contrast in digital images. Unlike global methods, CLAHE operates locally, dividing the image into small regions (e.g., 8x8 pixels) to apply contrast transformations independently. This is especially beneficial in images with variable illumination or contrast. The technique seeks to redistribute pixel intensity levels to achieve a more uniform histogram, expanding the dynamic range and revealing hidden details. After dividing the image into blocks, CLAHE calculates the histogram of each one, applying a contrast limit to avoid excessive noise amplification. Finally, bilinear interpolation is used between the blocks to ensure smooth transitions and avoid visual discontinuities in the enhanced image.

Chapter 3

Design and Implementation

This chapter elaborates on the solution developed for the problem. The characteristics of the crops, image capture, and the general system architecture, as well as the functioning of each stage, are detailed.

3.1 System Architecture

The proposed solution was implemented with an inference *pipeline* that can be seen in detail in figure 3.1. This system combines an initial preprocessing stage, the CV model instance, and post-processing of detections to identify lines and calculate density.

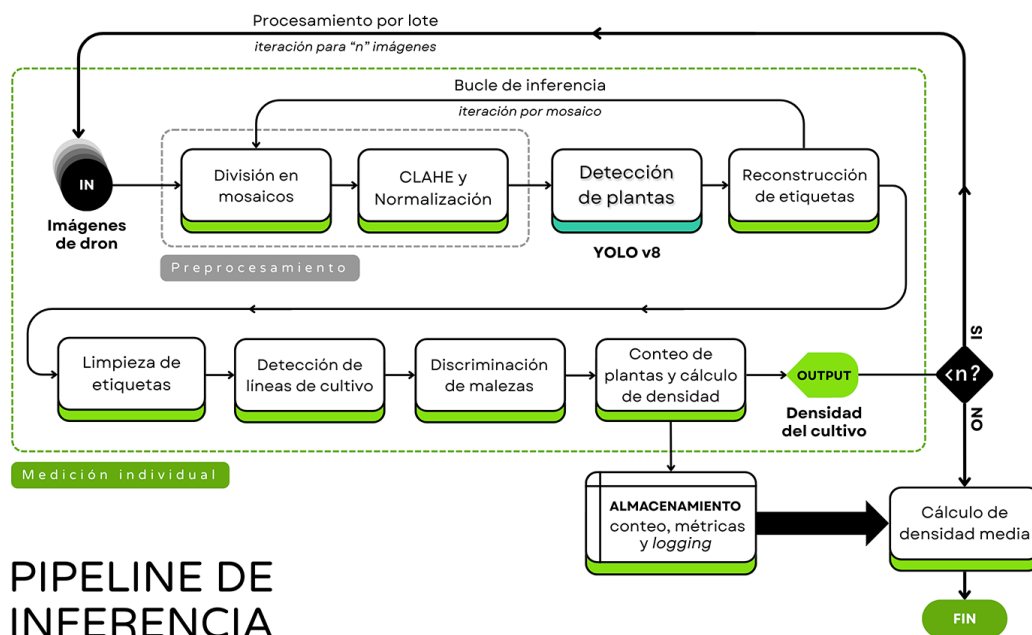


FIGURE 3.1. Inference pipeline diagram.

3.2 Image Capture

The dataset was built from images provided by the client. As it is a specific solution for a particular productive environment, it responds to the particular characteristics of the crops treated, the geographical location, and the specific needs of the client for whom it was designed.

3.2.1 Crop Characteristics

The images were captured at two Adecoagro company farms, located in the province of Corrientes, Argentina (table 3.1); sown between August 19 and November 5, 2024, with different varieties of rice: IRGA 424, IRGA 424 CL, IC 126, IC 109, IC120, IC131 and Kira.

TABLE 3.1. Information about the farms.

Farm	Location	Coordinates	Plots
Doña Marina	Yahapé, Corrientes	27° 26' S / 57° 40' W	209, 503, 212
La Verde	San Isidro, Corrientes	29° 43' S / 57° 28' W	211, 212

3.2.2 Drone Flight Plan

Initially, exploratory flights were carried out as shown in table 3.2, to capture images of different plant development stages (early and mid) at different heights (ranging from 1 to 10 m in height, at 0.5 m intervals) and angles of 30°, 45°, 50°, 60° and 90° (measured with respect to the horizontal ground plane).

In total, 4 stops were made in different sections of plot 209 (early development) and 503 (advanced development), and an additional one for plot 212 (flooded with early development). For each stop, 2 to 4 representative points were selected as samples, and photographs were taken for the mentioned heights and angles. Additionally, 1 or 2 sampling rings were incorporated per image, with which a traditional manual count was performed to use as *ground truth*.

TABLE 3.2. First image capture (2/10/2024).

Quantity	Plot	Height (m)	Angle (°)	Characteristic
140	209	1 to 10	90	Early development
112	503	1 to 10	90	Advanced development
13	209 503	1 to 5	60	
13	209 503	1 to 5	30	
12	209 503	1 to 5	45	
3	212	5 and 10	90	Flooded
2	209	1.5	50	

Subsequently, the quality of the captured images was verified by organizing them into folders by height and angle. A total of 52 defective images were discarded, in which the following problems were detected:

- difficulty focusing on low-altitude images (1 m),
- poor definition in some higher-altitude images (> 5 m),
- inability to capture useful images in strong wind conditions.

An *a posteriori* count was performed for the rings, which was compared with the *a priori* count to calculate the Root Mean Square Error (RMSE) and R^2 from equations 3.1. As can be seen in figure 3.2, from 6 m onwards, image resolution (pixel/cm) becomes a critical factor that makes correct plant identification impossible.

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{n}} \quad R^2 = 1 - \frac{SC_{res}}{SC_{tot}} \quad (3.1)$$

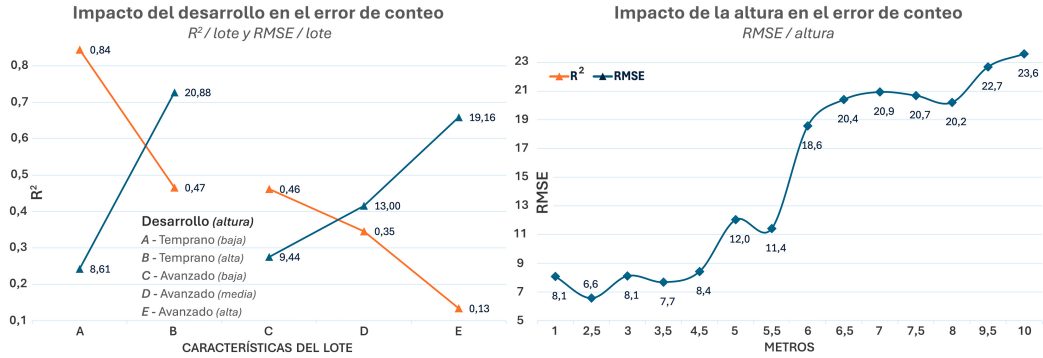


FIGURE 3.2. Error in plant count as a function of height.

Figure 3.3 shows the importance of performing the count at an early stage of plant development to ensure a good result. As can be seen, only for plot 209 was an adequate count obtained at 2 and 4 meters. For plants in an advanced stage of development, it becomes more difficult to recognize overlapping plants. This is why it was decided to work with images at 3.5 m, as they offer the best balance between detail resolution and covered crop area.

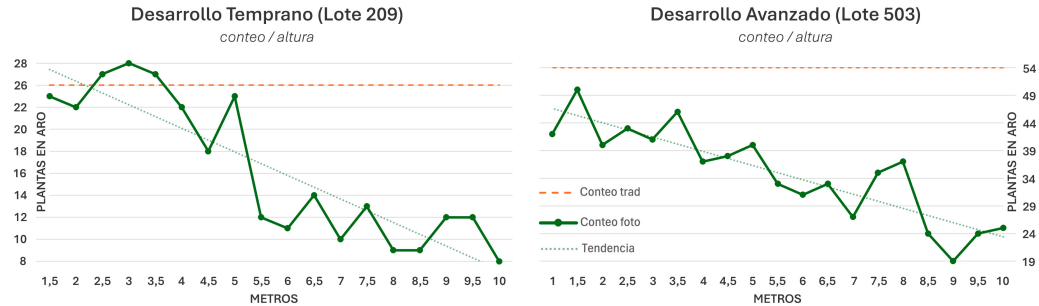


FIGURE 3.3. Count error by plot as a function of height.

Then, based on these results, it was decided to perform flights at lower altitudes (between 3 and 4 meters) to achieve a good balance between plant number and pixel resolution. In the Appendix C the flight protocol sent to the client to capture the dataset images is included.

In a second stage, the client provided 63 additional images of plants in late development (plots 211 and 212), with heights of 5, 10, 15 and 30 m and angles of 90° and 60°, as shown in table 3.3. 22 of the samples include the measurement ring and 4 images have the characteristic of including personnel footprints (same sector at different heights). Due to the nature of the problem, only development tests were performed with the lower altitude (5 m) angled and zenithal images, to carry out some tests during development.

Finally, these images were not incorporated into the training dataset since, due to the development stage, the plants were overlapping and reliable labeling was not possible.

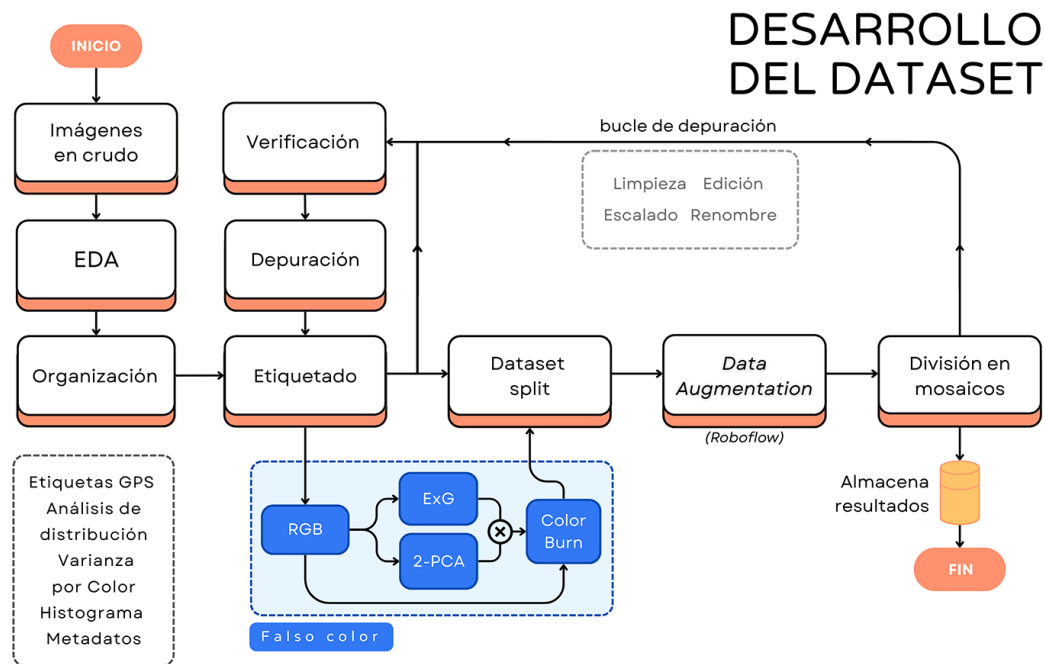
TABLE 3.3. Second image capture (12/12/2024).

Quantity	Plots	Height (m)	Angle
11	211 212	5	90°
12	211 212	10	90°
11	211 212	15	90°
10	211 212	30	90°
18	211 212	5 to 30	60°

3.3 Dataset Construction

Within the framework of this work, 24 images of plants at different development stages, with particular flight heights and camera angles, were labeled. During the development of the dataset, a total of 49 285 *bounding boxes* were assigned, demarcating rice plants, *clusters* of plants and foreign objects.

A set of *scripts* was developed to facilitate data analysis and processing, labeling, image preprocessing, *data augmentation* and debugging. These can be seen in detail in figure 3.4.

FIGURE 3.4. Diagram of *scripts* for dataset development.

The final dataset consists of 14 images with a zenithal view of rice crops in early (Plot 209) and advanced (Plot 503) development stages, taken at a height of 3.5 meters with a 90° angle. They are high-resolution color images, with dimensions of 5472×3078 pixels, in JPEG format with 3 RGB color channels, without an alpha channel. Each file weighs approximately 5 MB.

The solutions proposed by this work are based on two alternative datasets:

- Color images: composed of the original RGB images.
- False-color images: vegetation indices (*Excess Green*) are implemented.

3.3.1 Classes and Labeling Strategy

Although it is a binary classification problem (plant/background), given that these are images with a large number of labels (~ 2000 per image), it was decided to adopt a labeling strategy with four classes: `plant-weed`, `plant-occluded`, `plant-cluster` and `intruder`. This was done with the aim of visually recognizing problematic areas of the images and allowing for the future implementation of other complementary strategies that help in solving the counting problem.

Function of each class:

- `plant-weed`: these are the ideal labels, identifying all those plants that are clearly visible and can be labeled with a high level of confidence.
- `plant-occluded`: indicates those labels where there is a low level of labeling confidence (hidden plants, overlapping, covered by stubble, etc.). The main objective is to quickly identify problematic areas.
- `plant-cluster`: identifies areas with a high agglomeration of plants (called *clusters*) where correct labeling becomes practically impossible due to the high level of overlap. It was added with the idea of being able to apply some heuristic during counting or train models in parallel that would detect the *clusters* and apply some strategy (these solutions have not been implemented for the present work).
- `intruder`: identifies the few foreign objects found in the images (generally some type of weed or inorganic material). It was used to track these and quickly locate them.

For the training of the models in this work, the classes “`plant-weed`” and “`plant-occluded`” were unified under a single class, while the classes “`plant-cluster`” and “`intruder`” were omitted. That is, the YOLO model was trained to perform single-class detections (plants).

File Name Management

To images labeled with Roboflow, an internal 32-character ID (consisting of alphanumeric characters) is automatically added at the end of the original file name, after the “`rf`” tag.

Along with the image file, a text file with the labels is stored. The images are stored inside the folder `images` and the labels in the folder `labels`.

As an example:

- Input (unlabeled image): `209_101_24.jpg`
- Output (labeled image):
 - `images/209_101_24_JPG.rf.8d87d8429...cc299d413.jpg`
 - `images/209_101_24_JPG.rf.8d87d8429...cc299d413.txt`

3.3.2 Data Augmentation

For the training set, *Data Augmentation* of $3\times$ was applied with the Roboflow tool, to generate two new images for each existing one. In table 3.4 the details of the transformations applied can be seen. The criterion used to define the parameters

was to obtain realistic transformations, similar to the images the model might encounter in production.

1-Stage Data Augmentation Strategy

The process consisted of spatial transformations (rotation, mirroring, and cropping) and color transformations (saturation, brightness, and exposure). Additionally, a slight blur was incorporated to aid in detection in out-of-focus images, a problem identified during dataset construction. A small component of random noise was also added, which helps in model generalization, making it more resilient to sensor noise or minor imperfections.

TABLE 3.4. Roboflow Transformations.

Transformation	Parameters
Brightness	$\pm 25\%$
Saturation	$\pm 30\%$
Exposure	$\pm 5\%$
Rotation	$\pm 90^\circ / 180^\circ$
Mirroring (<i>Flip</i>)	H/V
Cropping (<i>Zoom</i>)	up to 30%
Blur (<i>Blur</i>)	up to 2 px
Noise (<i>Noise</i>)	up to 0.1%

Application for the False-Color Dataset

Alternatively, for the false-color dataset, some transformations (saturation, brightness, and blur) were removed so as not to limit the result achieved during image processing. The applied transformations were:

- Exposure: $\pm 5\%$
- Mirroring (*Flip*): H/V
- Rotation: $\pm 90^\circ / 180^\circ$
- Cropping (*Zoom*): up to 30%
- Noise: up to 0.1%

2-Stage Data Augmentation Strategy

Subsequently, it was decided to apply a more elaborate 2-stage strategy, to achieve a 9x increase in the number of images (also with Roboflow).

1st stage:

- Brightness: $\pm 25\%$
- Saturation: ± 20
- Mirroring (*Flip*): H/V
- Rotation: $\pm 90^\circ / 180^\circ$
- Cropping (*Zoom*): up to 30%

2nd stage:

- Brightness: -5% (reduction)
- Saturation: $\pm 10\%$
- Exposure: $\pm 10\%$
- Gradual rotation: $\pm 15^\circ$
- Shear: $\pm 10^\circ$ H/V
- Cropping (*Zoom*): up to 10%
- Blur (*Blur*): up to 1.5 px
- Noise (*Noise*): up to 0.1%

Gentle Data Augmentation Strategy

Additionally, the Ultralytics YOLOv8 implementation incorporates by default a slight data augmentation with the Albumentations library that seeks to simulate real-world visual artifacts and improve model robustness by applying with a low probability ($p = 1\%$): Gaussian blur, median blur, grayscale conversion and CLAHE. Each parameter can be seen in detail in table 3.5. This option can be deactivated if the Albumentations library is uninstalled before starting training.

TABLE 3.5. Albumentations Transformations (YOLO default).

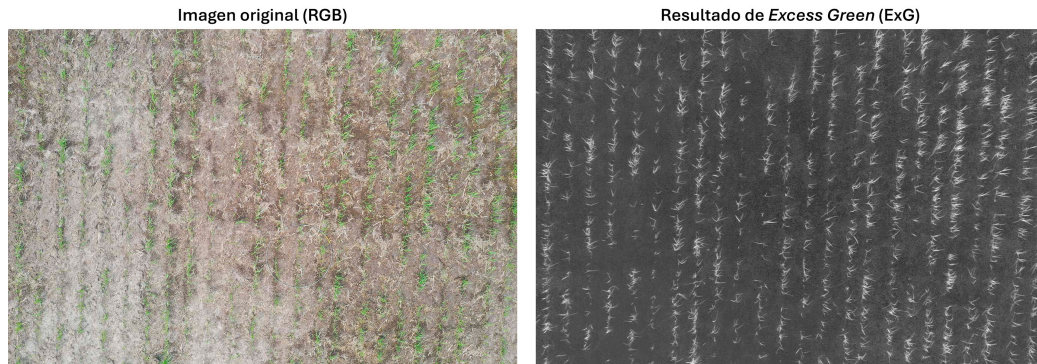
Transformation	Parameters
Gaussian Blur (GaussianBlur)	3 to 7 px
Median Blur (MedianBlur)	3 to 7 px
Grayscale (ToGray)	$0.299R + 0.587G + 0.114B$
CLAHE	clip_limit: 1 to 4 grid_size: (8, 8)

3.3.3 Application of Vegetation Indices (False-Color Dataset)

During dataset creation, various color space applications were explored, particularly the use of the Excess Green index.

Excess Green (ExG)

The *Excess Green* vegetation index was used to generate grayscale images that highlight the location of each plant within the image. The resulting image can be seen in 3.5. For its implementation, a Min-Max scaling of 0 to 255 was first applied to each color channel, and then the standard algorithm was processed.

FIGURE 3.5. Result of the *Excess Green* transformation.

Dimensionality Reduction (2-PCA)

The promising results in visually differentiating plants from the background using ExG images motivated their integration into the YOLOv8 architecture. However, since it traditionally processes 3-channel RGB images, the ExG image (in grayscale) could not simply be added as a fourth alpha (transparency) channel.

Given the limitation of having only two available channels (one for ExG), an analysis of the information distribution per channel in the dataset was performed by calculating an average histogram (figure 3.6).

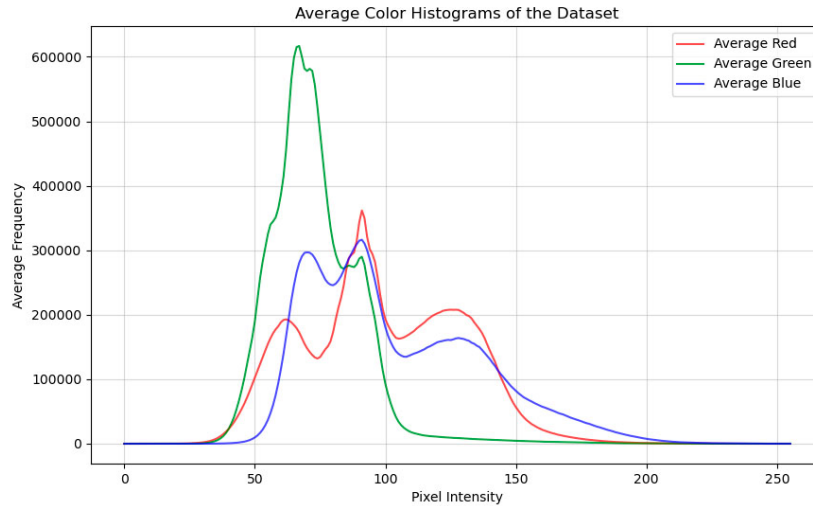


FIGURE 3.6. Average histogram per channel for the dataset.

Observing a significant contribution from all three original RGB channels, the complete elimination of one of them was ruled out. Instead, it was decided to apply a dimensionality reduction technique, specifically Principal Component Analysis (PCA), to reduce the information from the 3 RGB channels to 2 dimensions.

To implement PCA, the Scikit-learn (`sklearn`) library was used, and a model was trained with each image, to project the directions with the most variance into the new lower-dimensional space. To apply PCA to images, it is first necessary to transform the image vector from the standard format (height, width, channels) to the PCA input format (height x width, color); this was carried out with the Numpy library. Finally, Min-Max scaling needs to be applied to the PCA output so that the values again occupy the range $[0, 255]$.

As can be seen in figure 3.7, the second PCA component is very similar to the ExG result (with less contrast), while the first component is practically a representation of the background without plants.

False Color (FC)

A new RGB false-color image was generated by combining ExG with 2-PCA dimensionality reduction. Each grayscale image is assigned to an RGB channel.

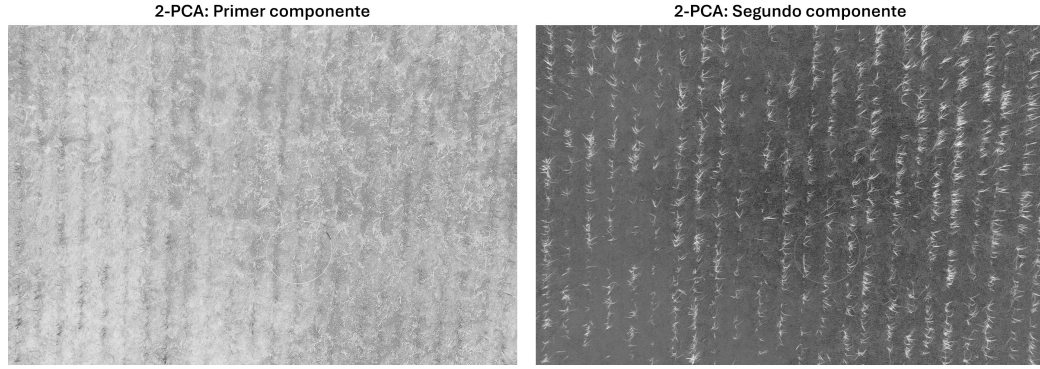


FIGURE 3.7. Result of dimensionality reduction with PCA.

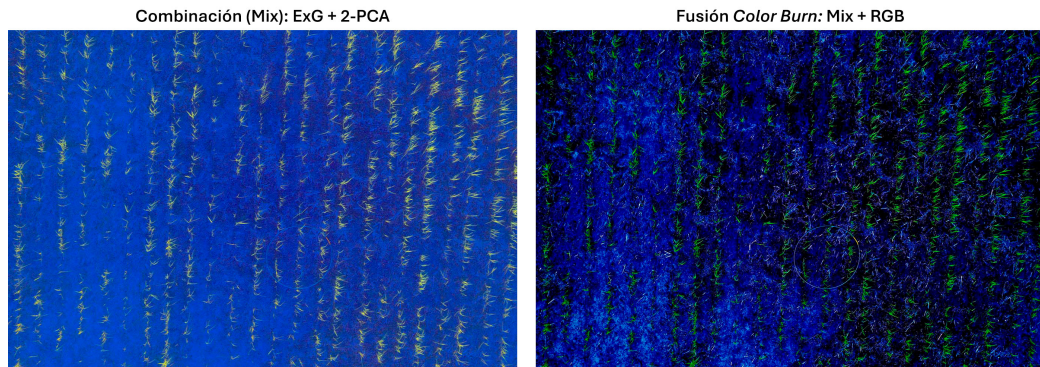
Different possible combinations for assigning each input to each channel (R, G and B) were studied, and the one that preserved the green color for plants was selected. This was done for practical purposes, being more recognizable to human vision, as it should not represent any difference for the (unpre-trained) model.

The resulting image is composed of:

- R channel: *Excess Green* (ExG)
- G channel: 2nd component (PC2)
- B channel: 1st component (PC1)

Additionally, this method preserves the property of plants to mostly occupy the green and red spectrum.

As can be seen in figure 3.8, with this transformation, it is much easier for the human eye to identify plants. It would be reasonable to expect a similar improvement for the model.

FIGURE 3.8. Result of *Color Burn* blending.

Color Burn

A final *color burn* stage was applied to merge the false-color image with the original (RGB), with the aim of preserving the high resolution of the drone photograph while incorporating the new information provided by the processed image (ExG+2-PCA), thus facilitating the detection task for the model.

Color burn darkens the colors of the base layer (original image) based on those of the blend layer (false-color image). This image blending is calculated with

the equation 3.2. The darker a color in the blend layer, the more it darkens the similar color in the base layer. This increases the contrast and saturation of the base layer's colors, giving them a more intense look, as seen in figure 3.8.

$$C_{out} = 1 - \frac{1 - C_{base}}{1 - C_{blend}} \quad (3.2)$$

Where:

- C_{out} is the resulting pixel value in the color channel.
- C_{base} is the pixel value of the base layer in the color channel.
- C_{blend} is the pixel value of the blend layer in the color channel.

This formula is applied per color channel (red, green, blue) independently, for pixels with normalized values in the range [0,1] (with Min-Max scaling, divide by 255).

3.3.4 Soil Images

Two images of unplanted soil were available, provided by the client. These were incorporated into the dataset with the aim of providing more background information and helping the model recognize possible areas without plants or with excessive stubble.

Upon detecting that the images contained some type of plant (possibly weeds or volunteer rice), it was decided to apply a censorship strategy for the regions of the image that contained green pixels. This was implemented using the *Excess Green* index with a threshold, to generate a mask of black pixels that would hide the plants, as can be seen in the image 3.9.

Then, a completely empty label .TXT file was generated, with the same name as the soil images.

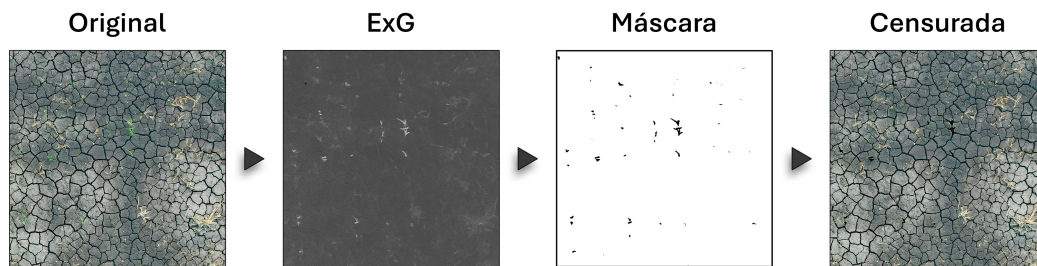


FIGURE 3.9. Soil image processing.

3.4 Image and Label Preprocessing

Image preprocessing is applied similarly to both the inference *pipeline* and the training *pipeline*, to ensure that the ETL (*Extract, Transform, Load*) process is the same. The only exception is the “label splitting” *script* that runs only during training. For inference, only the division of the image that will feed the model needs to be preprocessed.

3.4.1 Tiling

Each drone image was divided into square *tiles* to present the data in the format accepted by the YOLOv8 model. Although this version of the model supports a range of possible input image dimensions, these are always scaled to 640×640 px; for this reason, it was decided to work with those dimensions in the division to avoid loss of information.

The developed *script* allows processing images of any size and orientation, simply by indicating the dimensions of the output tile (always square), by default 640 px per side (named as constant S from now on).

The only size limitation is imposed by the file name format that allows indicating up to 100×100 tiles (with values from 00 to 99). This restriction is not relevant for the intended use case.

Offset (optional)

Optionally, a black border can be added before division so that an integer number of tiles (all of the same dimension S) are generated. In this way, the image is automatically centered (as seen in figure 3.10) so that the n tiles on the edges contain as much information as possible. If disabled, the tiles on the right and bottom edges would be smaller due to having fewer pixels than constant S .

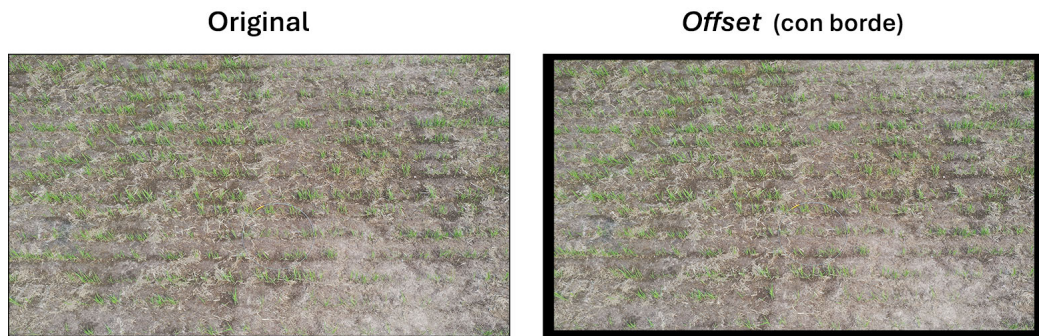


FIGURE 3.10. Original image and image modified with *offset*.

Although not essential for inference, it is recommended to keep this option activated as the model was trained with images with these characteristics and could negatively affect inference.

Image Division

For image division, RGB images are imported with the OpenCV library. The number of rows and columns to be generated (height and width, divided by S) is calculated. Then, the coordinates (x_0, y_0, x_1, y_1) at which the pixel matrix will be cropped are calculated. This is achieved by dividing the image dimensions (including the black border) by the corresponding row and column number multiplied by the constant S . This process is iterated for each of the tiles to be generated, storing them in an *array* of lists (`tiles[row][column]`) to preserve the data's matrix-like ordering. Each crop is stored on disk in the input format (`.jpg`) with OpenCV.

Label Division (for training only)

This process only applies to the training *pipeline* and allows distributing those labels from the labeled image that belong to different tiles.

Dividing labels is a critical process, since if the spatial correlation with the original image is lost, the model will not be able to learn from the images, as the labels will make no sense. For this reason, multiple verification stages were implemented to ensure that the number of input and output labels is consistent with the process performed.

To begin, the labels of the complete image are imported in .TXT format and a *DataFrame* containing the imported labels is created. As mentioned earlier, following the YOLOv8 PyTorch TXT format, labels are expressed in relative coordinates that indicate the location of each centroid and the dimensions of the *bounding box*:

```
class_id  center_x  center_y  width  height
0         0.187222  0.873889  0.014168  0.010380
```

This is why they are first converted into absolute values:

```
class_id  center_x  center_y  width  height
0         1024     2690     78      32
```

It is here that the black border previously defined for the image is incorporated, which shifts each label in the (x, y) direction of the *offset* vector. Next, the coordinates of the start points (x_0, y_0) and end points (x_1, y_1) for each *bounding box* are calculated:

```
class_id  x0      x1      y0      y1
0         3443   3503   2499   2541
```

All these transformations were performed in a matrix-like fashion to process all data as efficiently as possible.

Then, the label division loop is entered, which finds for each *tile* all those labels that belong to it and performs the division of the *bounding box* for labels shared with adjacent tiles.

A variable structure was generated to store all information in a unified way, through a list that stores for each tile a dictionary with the cropping coordinate tuple (x_0, y_0, x_1, y_1) , the *DataFrame* with the labels belonging to it (cropped) and the pixel matrix of the *tile*.

Through a meticulous debugging and testing process, the optimal functioning of this critical process was guaranteed.

Finally, the matrix of tiles is traversed again to retrieve the data stored in the variable structure, convert the *bounding boxes* back to labels and transform from absolute to relative values. All labels are grouped into a single line of text and stored in the new .TXT file generated for the tile.

Since it is a process with multiple nested loops, it usually takes about a minute to complete execution. For this reason, a progress bar and a detailed *output (verbose)* were incorporated to indicate the user the level of progress.

File Name Management

To maintain consistency in file management, it was decided to add the tile identifier before the Roboflow code, that is, next to the original image name.

The following format was adopted: `tile`, row number, `x`, column number.

An example is shown below:

- Input:
`209_101_24_JPG.rf.8d87d842911d...37ccc299d413`
- Output:
`209_101_24_JPG_tile01x06.rf.8d87d842911d...37ccc299d413`

The corresponding extension (`.jpg` or `.txt`) was added depending on whether it was the tile or its labels.

For the inference *pipeline*, each of the generated tiles is stored in a temporary local folder (`data/inference/image_name`), which is automatically deleted once the plant counting is finished.

3.5 Adaptive Equalization (CLAHE)

It was decided to apply adaptive equalization to each tile to limit the risk of variability of images captured in productive environments, where brightness, contrast, exposure or saturation can vary widely depending on weather conditions, time of day and season.

For each *tile*, the intensity histogram is calculated, the contrast limit (*clipping*) is applied to avoid excessive noise amplification and values exceeding that threshold are redistributed to other *bins* of the histogram. Finally, a transformation function based on the modified histogram is calculated and applied to the pixels within the *tile* to adjust its contrast.

Different configuration values were tested and those that helped differentiate plants from the background better were chosen: limit 2 (`clip limit`) and grid size 16×16 (`grid size format`).

3.6 Normalization (Range Scaling)

YOLOv8 automatically applies scaling normalization, where pixel values are scaled to the range of $[0, 1]$ by dividing them by 255. This technique assumes the minimum (0) and maximum (255) values of the original range as constant for all images. That is why it is also known as $[0, 1]$ normalization or 0 to 1 scaling.

3.7 False Color Preprocessing (Optional)

In the case of working with the false color (FC) dataset, an initial preprocessing stage is added before tiling to transform the RGB images to the new color space. The rest of the *pipeline* process continues unchanged.

As can be seen in figure 3.11, the aforementioned transformations are applied sequentially:

1. *Excess Green* (Grayscale)
2. Dimensionality Reduction: RGB $\rightarrow$ 2-PCA (Grayscale)
3. Combination: 2-PCA + ExG (False Color)
4. Color Burn Blending: False Color + RGB (False Color)

In this way, an alternative dataset is generated that manages to highlight plants and obscure the background by combining the advantages provided by the excess green index, without loss of information (due to PCA application) and without loss of resolution in the image (due to blending with *color burn*).

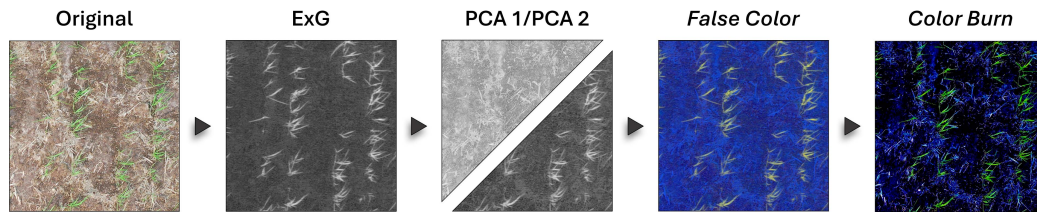


FIGURE 3.11. Process of generating false color images.

3.8 Object Detection Model

In this stage, inference is performed for each of the tiles stored in `data/inference/image_name`.

A total of 90 different models were trained with an estimated total GPU usage time of 270 hours, for a VRAM utilization between 75% and 95%. Table D.1 can be consulted for details on the hardware used.

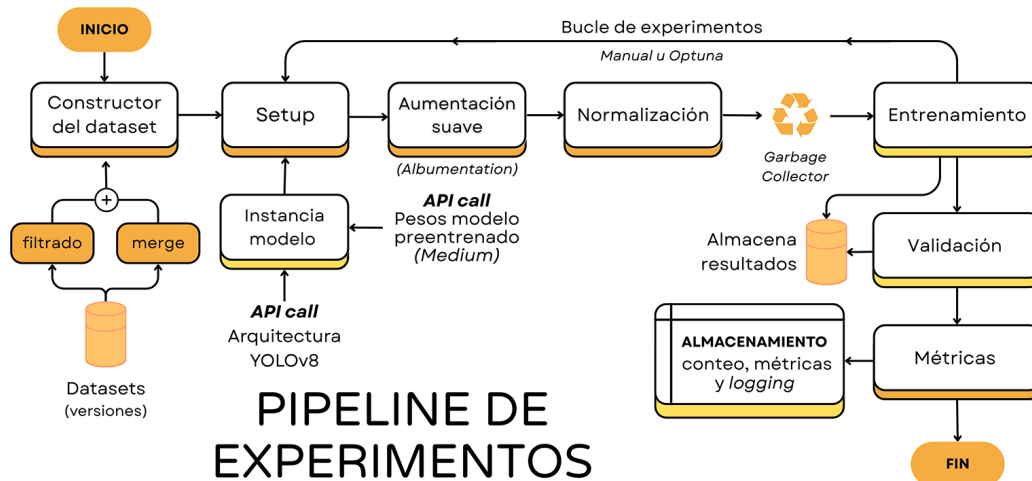
3.8.1 Data Split

The final dataset consists of a total of 896 tiles generated from 14 high-resolution zenithal images of crops captured at a height of 3.5 m and 2 images of unplanted soil. A split of 57.8% for training data, 24.1% for validation and 18.1% for testing was used. By including *Data Augmentation* $9\times$, a total of 3542 tiles were reached for the training set.

3.8.2 Model Training

All experiments for different models were trained through the Ultralytics API (importing the library of the same name). Given that the training sessions were performed in the cloud, the *notebook* is linked to a shared folder in Google Drive that contains the different versions of the dataset used for the experiments, as can be seen in figure 3.12. The data to be used is copied to the virtual machine to be accessed efficiently.

Then, the YOLO library that defines the model architecture is imported, and the pre-trained weights for the corresponding version (mostly YOLOv8 Medium) are loaded by downloading the PyTorch file (`yolov8m.pt`) from Ultralytics. For some experiments, the model was initialized with random weights through the YAML file (`yolov8m.yaml`).

FIGURE 3.12. Diagram of *scripts* for experimentation.

Before starting training, a *Garbage Collection* instance (`gc.collect()`) is executed to clear memory, since resources in Colab are limited and session interruption is common if unused resources are not freed. The environment variable `PYTORCH_CUDA_ALLOC_CONF` is also set with `expandable_segments:True`, to try to reduce VRAM fragmentation and thus use the GPU more efficiently.

Thus, the training of the model in question begins with the hyperparameter configuration to be explored. For most training sessions, a training time limit was set according to the available resources and the maximum `batch size` was defined to make the most of the GPU.

The complexity of this architecture is a size to consider, since, for the *Medium* size, approximately 25.9 million parameters are adjusted per epoch. Even despite having applied the *Transfer Learning* technique, it was necessary to perform long training sessions to give the model time to learn the characteristics of the new images, given that the YOLO architecture was pre-trained for a dataset (COCO) with a quite different visual domain. A standard training for the developed datasets took between 2 and 4 hours to reach its best epoch.

For this reason, it was decided to give the model the possibility of performing long training sessions: the number of epochs was limited (from 500 to 1000) or the execution time (1 to 4 h); but always with *Early Stopping* according to the objectives of the experiment, so that the training would be interrupted if no considerable improvements were made.

Once training is finished, it is crucial to immediately make a backup copy in the cloud, since, if the session is interrupted, all the work done is lost. To mitigate this risk, a *script* was implemented that automatically stores all training results (weights and graphs) in Drive.

3.8.3 Model Validation

For the validation process, the weights of the newly trained model (`weights/best.pt`) are loaded into the YOLO architecture, as can be seen in figure 3.13. As with training, the largest *batch size* supported by the GPU was used. The `verbose` and

`save_json` functions were activated to store as much information as possible about each training.

A *script* was generated to extract the confusion matrix from the results delivered by YOLO and thus calculate the metrics of interest: *Accuracy*, *Precision*, *Recall*, *F1-Score*, *F_{1/2}-Score* and *G-measure* (which will be explained in detail in the next chapter).

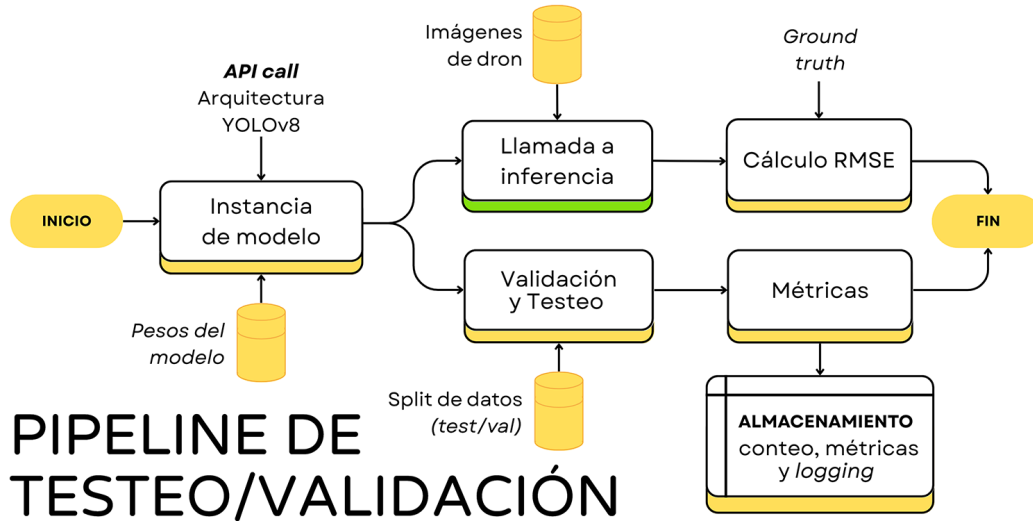


FIGURE 3.13. Diagram of *scripts* for validation and testing.

Additionally, validation tests were performed where different hyperparameters were manually defined or explored through stochastic searches with Optuna.

3.8.4 Inference: Plant Detection Module

For the inference *pipeline*, shown in figure 3.1, a YOLO model is instantiated and loaded with the weights of the best trained model (or the model in question to be tested). All tiles generated during image division (stored in: `data/inference/image_name`) are imported.

Sequentially, inference is performed for each tile and the *bounding boxes* of the objects detected by the model are stored in various formats. First, the `.TXT` file with predictions in YOLOv8 PyTorch format (contains class and coordinates of each object) is saved.

Additionally, for each inference, the visualization with *bounding boxes* generated by YOLO is stored (this option is deactivated in production) and a `.JSON` file containing detailed information for debugging:

- Image name.
- Possible class names.
- Each of the detections with:
 - Box coordinates (x_0, y_0, x_1, y_1) .
 - Confidence.
 - Predicted class.

- Image dimensions.
- Speed (for the YOLO model):
 - Preprocessing.
 - Inference.
 - Postprocessing.

These files are stored under the original name preceded by the prefix “inference.” in the same directory (`data/inference/image_name`).

For the *testing* stage, a Jupyter notebook was created that facilitates iteration over multiple models to be compared. In this, inferences are performed with each one for the same image, predictions, metrics and log files are saved for debugging. This *notebook* presents the multiple predictions, measures the inference time for each model and indicates the percentage of progress of the process. At the end, it presents the total inference time for a complete image (all tiles) and average inference time/tile.

3.9 Prediction Post-processing

Finally, the tiles and predictions (stored within `data/inference/image_name`) are unified to generate the count for the complete image and thus calculate the density of plants per hectare.

3.9.1 Label Reconstruction and Cleaning

The inverse process of label splitting is performed with the objective of unifying all detections made during multiple inferences. First, the tile grid is reconstructed to iterate over the matrix of *tiles*, assigning to each inference the corresponding coordinates in the complete image (includes the *offset* of the black border).

Each of the `.TXT` files with the predictions for the corresponding tile is opened and imported into a *DataFrame*. This is stored within a dictionary along with the position of the tile matrix (row/column).

For each *tile*, the labels are transformed from relative to absolute values and the coordinates of the *bounding boxes* (start and end point of the box) are calculated. A new empty *DataFrame* is created in which all detections found for each tile are concatenated. Being a critical process, various verification steps were implemented to ensure that there is no data loss during the process.

Once all the *bounding boxes* are unified, they are converted back into labels (center point, width and height) and transformed to relative values, to complete the export to the final `.TXT` file.

3.9.2 Crop Line Detection

Given the characteristics of the client’s fields, it is assumed that all furrows have a practically parallel distribution. The system is not designed to analyze patterns in curvilinear furrows or variable orientations (oblique lines).

To begin, the general slope of the crop lines is estimated using polynomial least squares fitting (with `polyfit`), in this case for first-degree polynomials as they

are straight lines. A line $f(x) = mx + b$ is fitted to the general set of points and the inclination angle $\theta = \arctan(m)$ is calculated.

Then a rotation matrix (3.3) is applied to ensure that the crop lines are horizontally aligned (a necessary condition for the proper functioning of DBSCAN). As seen in equation 3.4, the position vector $\vec{p}_{(x,y)}$ is multiplied by the matrix.

$$R(-\theta) = \begin{bmatrix} \cos(-\theta) & -\sin(-\theta) \\ \sin(-\theta) & \cos(-\theta) \end{bmatrix} \quad (3.3)$$

$$\vec{p}_{rotated} = R(-\theta) \cdot \vec{p}_{(x,y)} \quad (3.4)$$

Next, the *clustering* algorithm DBSCAN is applied, with a distance tolerance of 0.01 and a minimum of 5 points per *cluster*. This gathers all those points that could belong to the same group, assigning them an index (positive integer) and marks the points that could not be assigned with value -1.

Although DBSCAN could identify *clusters* with any orientation, more precise results are obtained by aligning all points with a single axis, as this simplifies the search for neighbors to a single dimension. This makes it more efficient and effective, as can be seen in figure 3.14.



FIGURE 3.14. DBSCAN *cluster* assignment.

Subsequently, `polyfit` is used again to find the line that best fits each *cluster* and a dictionary is created with the coefficients for each group.

At this instance, all those lines that overlap within a certain tolerance margin are verified and their points are reassigned to the same cluster (due to being false positives in DBSCAN's assignment).

Finally, the distance to the line is measured for all those points that could be grouped and they are tentatively assigned to the closest *cluster*. In figure 3.15 the result of this process can be seen.

3.9.3 Weed Discrimination

“Weed” is considered to be all those points that are positioned with a distance greater than the threshold perpendicular to the crop line (whether red rice, volunteer rice or other invasive species). Empirically, it was determined that with a threshold of 4 standard deviations it is possible to filter out those plants that are between furrows, without losing correct detections due to high dispersion. Each point that exceeds this threshold is marked as “potential outlier”.

Next, the possible *outliers* are assigned to the immediate adjacent group to verify if they were erroneously assigned to a neighboring *cluster* and the distance to the new line is checked. Only those objects that were identified as *outliers* for both lines will effectively be “weed”. An example of weed detections can be seen in figure 3.16.

It is important to note that each point indicates the centroid of the *bounding box* (not the plant stem), so this dispersion is generally due to the plant lying on the ground. This aspect worsens as the plant size increases, as it progressively deteriorates the model’s ability to detect weeds.

3.9.4 Plant Counting and Density Calculation

Finally, a count is performed of those points that were not marked as “weed”. It is also verified that no objects were lost during the filtering process.

The area covered by the image as a function of height is a known parameter, which was experimentally measured during the construction of the dataset. In table 3.6 the reference values considered for the heights used in the experiments can be seen. If required, for working at another height or with different resolution, it is possible for the user to introduce a personalized value for the area calculation.

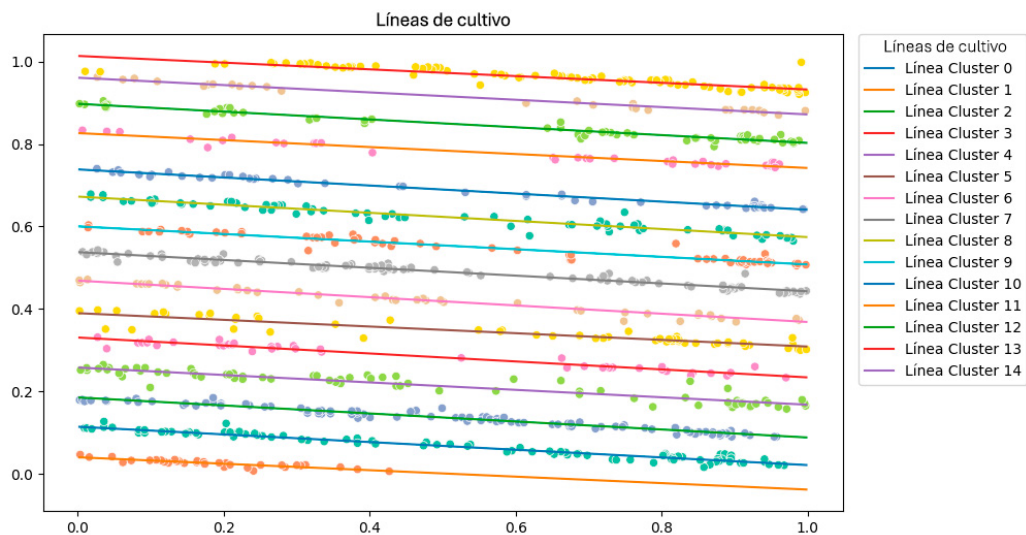


FIGURE 3.15. Crop line detection.

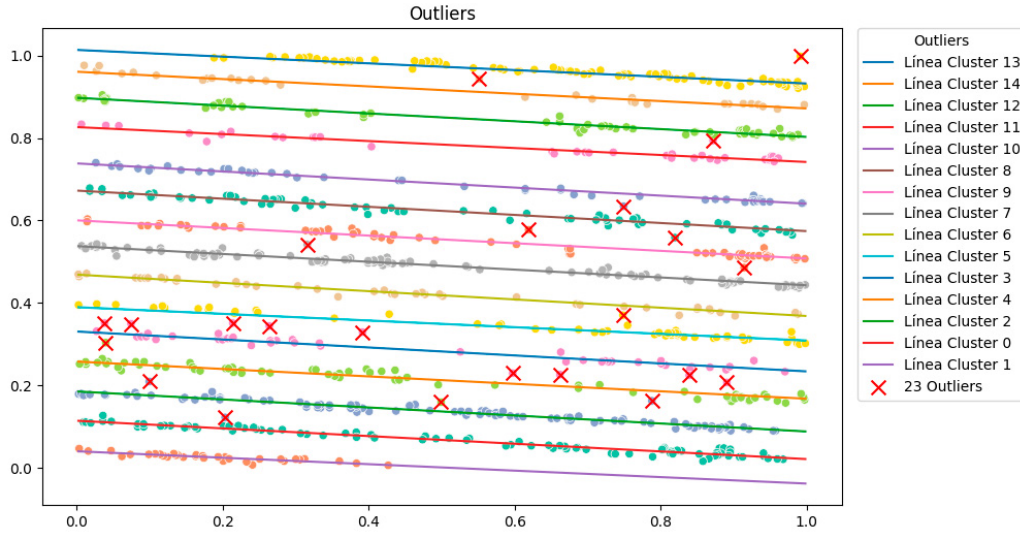


FIGURE 3.16. Weed detection (outliers).

TABLE 3.6. Values of the area covered by the photograph.

Height (m)	Resolution (cm/px)	Area (m ²)
3.5	0.082	13.35
5	0.120	28.75

Given this constant, the density of plants per m² and per hectare is calculated, with equation 3.5:

$$density_{ha} = \frac{n_{plants}}{area} \times 10\,000 \quad [pl/ha] \quad (3.5)$$

Both density values are delivered by console to the user, along with the count and the number of weeds. For each processed image, a *log* file is generated and the counting results are stored in a *CSV* file.

3.10 Batch Processing

For *batch* processing, the counting process is sequentially repeated for each sample (n images from the same batch). As explained previously, the result of each individual count is stored on disk. Then, the data from the *CSV* corresponding to the processed images are imported and the arithmetic mean (average value) of the density per hectare is calculated with equation 3.6:

$$\overline{D}_{ha} = \frac{1}{n} \sum_{i=1}^n d_{i,ha} \quad (3.6)$$

3.11 Metric Suite

In meetings with the client, it was reported that a measurement that overestimated plant density implied a greater risk for the company, given that it prevented palliative measures from being taken during cultivation, with the problem only being detected at harvest time, which generated a monetary loss due to decreased yield.

On the contrary, a downward-biased measurement would generate greater attention to verify the situation and act accordingly. It was then determined that it was a priority to limit False Positives (FP) over False Negatives (FN).

For this reason, initially it was decided to choose *Precision* as a key performance indicator (KPI) to quantify the proportion between True Positives (TP) and False Positives (FP), as can be seen in equation 3.7:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (3.7)$$

Subsequently, it was decided to construct a personalized metric that better suited the nature of the problem. It was decided to work with F_β -score, with equation 3.8, and assign it a value $\beta = \frac{1}{2}$ to give greater weight to *Precision* than to *Recall*, and thus try to reduce FPs. The final metric can be found in equation 3.9.

$$F_\beta = (1 + \beta^2) \frac{\text{Precision} \times \text{Recall}}{(\beta^2 \times \text{Precision}) + \text{Recall}} \quad (3.8)$$

Substituting $\beta = \frac{1}{2}$:

$$F_{1/2} = \left(1 + \left(\frac{1}{2}\right)^2\right) \frac{\text{Precision} \times \text{Recall}}{\left(\left(\frac{1}{2}\right)^2 \times \text{Precision}\right) + \text{Recall}} = \frac{\frac{5}{4} \text{Precision} \times \text{Recall}}{\frac{\text{Precision}}{4} + \text{Recall}} \quad (3.9)$$

Complementarily, the *Fowlkes-Mallows index* (*G-measure*) was incorporated, from equation 3.10, which is the geometric mean between *Precision* and *Recall*.

$$\text{G-measure} = \sqrt{\text{Precision} \times \text{Recall}} \quad (3.10)$$

Unlike F_β (harmonic mean), which heavily penalizes low values, the geometric mean is more permissive with extreme values. A model that focuses on achieving high precision—to minimize false positives—could have a slightly lower *recall*, which would be penalized by a metric like *F1-score*, even if it represented an improvement within that context. *G-measure* is useful when one wants to maintain a balance between both values, without achieving excellent precision at the cost of detecting very few real objects.

Furthermore, *Mean Average Precision* (mAP) was used as an evaluation metric for object detection tasks. Unlike simple precision, which evaluates the accuracy of predictions in general, mAP considers both precision and *recall* at different confidence levels or thresholds, and then averages these values, as can be seen in equation 3.11. Given that determining the exact position of the *bounding boxes* was not a decisive factor, as the detection itself was important, mAP was used

for an Intersection over Union (IoU) threshold of 50% (named in the context of YOLO as mAP@0.5).

$$mAP = \frac{1}{k} \sum_i^k AP_i \quad (3.11)$$

As a complementary measure, traditional metrics (*Accuracy*, *Recall* and *F1-score*) were also used as a reference during testing, but were not considered for critical development decisions.

Chapter 4

Essays and results

In this chapter, the results obtained for the different experiments carried out during the development of the work are presented.

4.1 Model tests

In total, 35 experiments were carried out with different hyperparameter configurations and evaluated datasets. A total of 90 models were trained, for which 460 validation/testing trials were performed and 2214 inferences were made to process measurements on 42 images with different characteristics.

Up to experiment 15, mosaics of 240×240 px were used. From then on, the resolution was changed to 640×640 px, as it is the input format requested by YOLOv8.

The experiments carried out can be grouped into:

- Dataset construction.
- Tests with *Data Augmentation*.
- Experiments with vegetation indices (false color).
- YOLO architecture:
 - Versions: v8, v9, v11, v12.
 - Sizes: *Nano / Medium / Large / Extra-large*.
- *Fine-Tuning (freezing)*:
 - Random initialization (without pre-training)
 - Full *Fine-Tuning*: pre-trained model without freezing,
 - Freezing part of the *Backbone* (8 layers),
 - Freezing the entire *Backbone* (10 layers),
 - Freezing part of the *Head* (12 layers).
- Experiments with hyperparameters: *batch size*, number of epochs, *early stopping*, learning rate, *momentum*, *multi-scale training*, *dropout* and L2 regularization (*weight decay*) and loss function gain (*box loss*, *classification loss* and *distribution focal loss*).

- Validation tests: *confidence*, IOU and agnostic NMS (*agnostic non-maximum suppression*).
- Inference experiments (*testing*).

Details of the hardware used can be found in appendix [D](#).

4.1.1 Tests with different architectures

Although design tests were performed with other YOLO versions (v9, v11 and v12), it was decided to work with version v8 because it offered the best balance between detection quality and training time. More recent versions are optimized for video processing by requiring less inference time. This did not represent an additional advantage for the problem presented here.

It was observed that more modern versions of YOLO achieved a higher mAP, but considerably increasing FPs. These models improve in spatial detection with bounding boxes, but metrics such as Precision and $F_{1/2}$ -score deteriorate. For these reasons, it was decided to use YOLOv8 as the base architecture.

In addition, experiments were conducted for different network sizes (*Nano*, *Medium*, *Large*, *Extra Large*). The change from *Nano* to *Medium* represented a considerable improvement (+25% in *Precision* and +17% for $F_{1/2}$ -score).

However, for larger scale models, improvements in these metrics were obtained (for *Extra*, *Precision* +15% and $F_{1/2}$ -score +10%), but with considerable increases in training (4.6x) and inference (2.7x) times and in the necessary hardware resources (both in training and inference); to achieve merely a +4% improvement in TPs. For these reasons, it was not considered an opportune trade-off to use models larger than *Medium* during development.

4.2 Dataset Construction

Preliminary experiments were conducted with small sets of labeled images at heights of 3.5 m and 5 m, with angles of 90° and 60° (with plants in early, mid, and advanced development). Pilot training runs were performed with these small datasets to compare the impact of different data types on model performance from an early stage.

During the labeling process, a total of 49 285 bounding boxes were placed within the four classes mentioned in the previous chapter. It can be seen in Table [4.1](#) that there is an approximate 6:1 ratio between the `plant-weed` (high confidence label) and `plant-occluded` (low confidence label) classes. The latter corresponds to potential problematic cases that might be difficult for the model to detect: overlapping, occluded, or extremely small plants.

Figure [4.1](#) shows that the labels are similar in shape and size, with a random distribution in the (x, y) plane and are not biased.

It is important to remember that this class differentiation was only used for labeling, due to the great complexity of the images, which contained an average of 2 thousand plants. At the time of training, the only classes used were `plant-weed` and `plant-occluded`, unified under the same class.

TABLE 4.1. Distribution of labels by class.

Class	Totals	
intruder	30	0%
plant-cluster	715	1%
plant-occluded	6647	13%
plant-weed	41 878	85%
Total labels	49 285	
Plants/img	2022	

4.2.1 Results with angled images

It was observed that labeling images with a 60° inclination was notably easier, as they allow for easier recognition of the plant stem. However, training tests showed an increase in false positives (+10%) compared to 90° images; this negatively affects the detection of true positives in similar proportions.

Images with 60° angles were not incorporated into the final version of the dataset as they did not allow for precise measurement of plant density.

4.2.2 Results for different flight heights

The initial observation, made during data classification, was verified. As flight height increases, although the covered area increases, allowing a larger portion

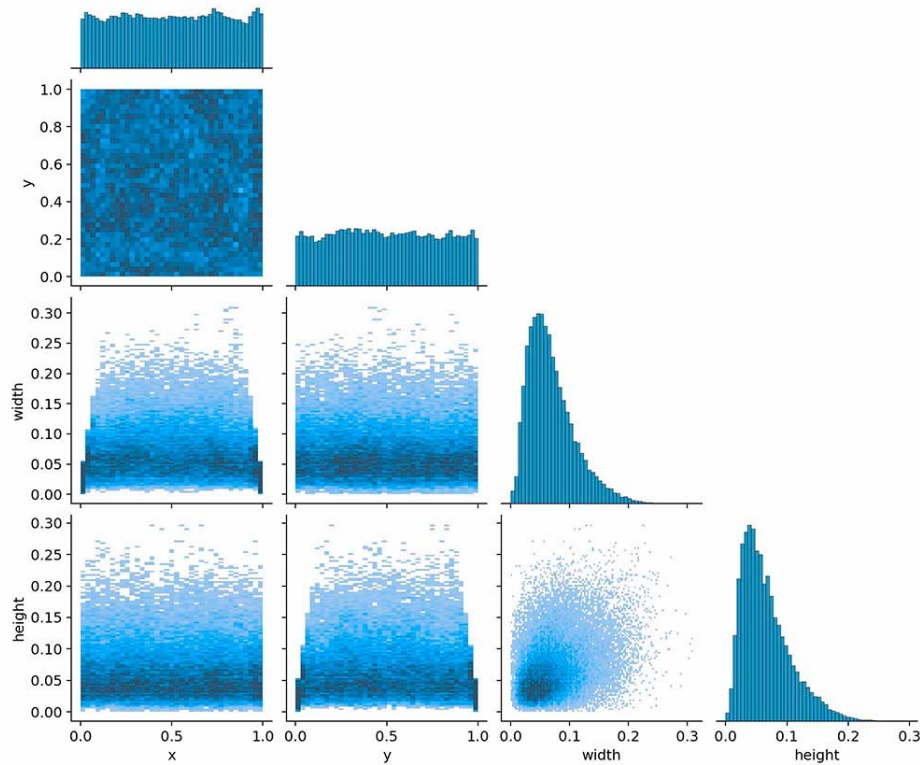


FIGURE 4.1. Label correlogram.

of the crop to be studied, camera resolution (measured in cm per pixel) also decreases. This means that the increase in coverage does not compensate for the loss of detail (see table 3.6).

Given that the rice crops studied here have small dimensions in early post-emergence stages (with seedlings between 1 and 5 cm tall), this implies that it becomes increasingly difficult for the model to recognize small objects as flight height increases.

Although for late stages of development it would be possible to work at a greater height, the rice plant has the characteristic of starting tillering between the third and fifth week of sowing. This means that the plant density increases as lateral stems (tillers) sprout, making it impossible to accurately determine the number of plants.

This was noticeable during the image labeling process, as it became extremely difficult to determine when it was a single plant and when it was several. Initial training tests yielded unfavorable results that evidenced the model's inability to recognize small objects from a high altitude. With small plants, it had 85.6% false negatives. For plants in advanced development, it achieved only 7.8% true positives, 24.2% false positives, and 68% false negatives.

As mentioned earlier, through experimentation with the models, it was verified that the optimal temporal window for counting ranges from one to three weeks, before the plant begins tillering.

4.3 YOLOv8 Configuration

During the development of the inference *pipeline*, different hyperparameter configurations for the YOLOv8 model and strategies for applying *transfer learning* and *data augmentation* techniques were experimented with.

4.3.1 Transfer learning

For *Transfer learning*, although the freezing of different numbers of layers of the architecture (including *backbone* and *head*) was explored, it was finally not applied for the definitive models as an improvement was observed when training the complete network. This was somewhat expected, as the images were from a domain far from that of the original dataset (COCO).

As can be seen in Table 4.2, the best balance between results and training time was achieved by loading the weights of the pre-trained model of medium size (*Medium*) and then continuing the training of the entire network (detailed as «Full Fine-Tuning (no freezing)»).

Although the randomly initialized model («No pre-train (random weights)») achieved slightly higher precision, by reducing FPs (-12.3%), it was also able to recognize fewer TPs (-5.6%) than the pre-trained model. In addition, it required 1.54x more training time.

In the case of the false color dataset, the use of the pre-trained model negatively impacted predictions. For this reason, random initialization (loading the `YAML` file) was used. This example clearly shows how, when working with images of

TABLE 4.2. Results for *Transfer learning*.

Experiment	Precision	Recall	F $\frac{1}{2}$ -score	mAP@0.5
No pre-train (random weights)	59.8%	54.0%	0.587	0.449
Freezing Backbone (8 layers)	58.8%	51.3%	0.570	0.433
Freezing Backbone (10 layers)	53.7%	57.0%	0.543	0.451
Freezing Head (12 layers)	56.7%	52.3%	0.557	0.450
Full Fine-Tuning (no freezing)	58.1%	60.3%	0.586	0.495

colors and shapes significantly different from the pre-training dataset, the model was unable to transfer learning to the new domain.

4.3.2 Hyperparameter tuning

The configuration used for the final models is described below.

Weighted loss function (*Loss function*)

YOLOv8 uses a complex loss function that combines a weighting between class detection (`cls`), bounding box coordinates (`box`), and *Distribution Focal Loss* (`dfl`). Hyperparameter exploration yielded the best results for an approximate 2 to 1 ratio between class loss values and the other two (for example: `cls=3 | box=1, 5 | dfl=1, 5`). This was achieved with the following configuration of the weights:

- `cls=1`, higher than the default value (0.5).
- `box=5`, lower than the default value (7.5).
- `dfl=1, 5`, default value.

Multi-scale training (`multi_scale`)

This was the hyperparameter that had the greatest positive impact on all study metrics. Multi-scale training helps detect objects of different dimensions by randomly varying the size of the input images. As a result, the model's generalization capacity is improved.

SGD Optimizer

Given the versatility of this YOLO implementation and the large number of hyperparameters, it was decided to use the automatic configuration for the optimizer, suggested by the *framework* based on the dataset.

Specifically, stochastic gradient descent (SGD) was used as the learning rate optimizer, with the following configuration: `optimizer: SGD(lr=0,01, momentum=0,9)`.

Momentum

Momentum is a parameter used in the optimizer to accelerate gradient descent and avoid getting stuck in local minima. Although attempts were made to configure it manually, YOLO always fixed it at 0.9 when using the SGD optimizer.

L2 Regularization (`weight_decay`)

Weight decay is a regularization technique that penalizes large weights in the model, which helps prevent *overfitting*. It was set with a value of 0.0015.

YOLOv8 automatically determines which groups to apply L2 regularization to:

- 77 weight groups were not subjected to *weight decay* (`decay = 0`).
- 84 weight groups were subjected to the established value (`decay = 0,0015`).
- For the 83 *bias* groups, regularization is not usually applied (`decay = 0,0`).

Dropout

Experiments were conducted with different levels of *dropout* and it was determined that incorporating a small amount of neuron "shutdown" during training offered a small improvement in metrics. This helps prevent the model from becoming too dependent on specific neurons and improves its generalization capacity.

The best results were obtained with a range less than 10%. For the final model, it was set at 5%.

Other hyperparameters explored

The Ultralytics YOLOv8 implementation is extremely robust and has been designed to automate most hyperparameter configuration.

In some cases, as with *momentum*, despite a value having been configured, YOLO ignored the initial configuration by finding a better alternative, given the nature of the task and the dataset.

In other cases, as happened with the *learning rate*, manual configuration did not offer better results. The *framework* automatically adjusts some hyperparameters that it "considers" more appropriate to complete learning optimally within the training time or epochs for which it has been configured.

Furthermore, being a single-class problem, the application of Agnostic NMS (*agnostic non-maximum suppression*) does not represent any change and is considered irrelevant.

4.4 Performance of the best models

Numerous models were trained with different datasets and hyperparameter configurations. As a result of this process, the following models were developed for the used dataset variants: color images (RGB) and false color (ExG+2-PCA+Burn). The training results are available in Table 4.3 and Figure 4.2.

TABLE 4.3. Model training results.

Metric	RGB ₈₆	FC ₆₈	Ratio
Data Augmentation	9x	3x	+300%
Training time (h)	0.5	3.5	+14.29%
Best epoch	17/18	61/359	3.6x
Box Loss	1.224	1.999	-38.79%
Class Loss	2.168	1.170	+85.25%
DFL Loss	1.296	1.442	-10.13%
True Positives	43.89%	40.00%	+9.73%
False Positives	23.02%	27.36%	-15.86%
False Negatives	33.08%	32.64%	+1.35%
mAP@0.5	0.554	0.474	+16.88%
Precision	0.656	0.594	+10.44%
Recall	0.570	0.551	+3.45%
F1-score	0.610	0.571	+6.83%
F _{1/2} -score	0.637	0.585	+8.89%
G-measure	0.612	0.572	+6.99%

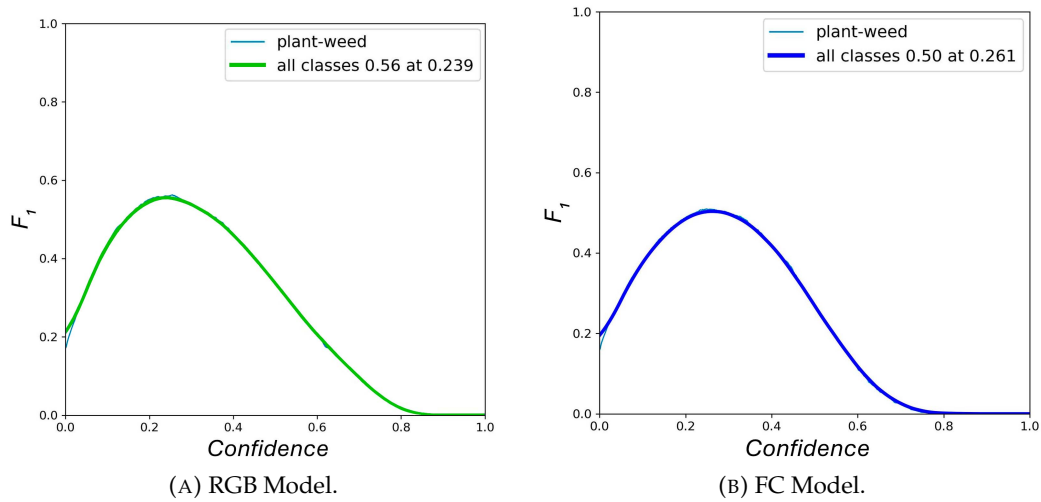


FIGURE 4.2. F1-score curves as a function of confidence.

For the color dataset, it was detected that the best results were achieved with a short training (around 30 minutes), since it is a pre-trained model. For longer training times, the model overfitted the training data (*overfitting*).

On the contrary, for the false color dataset, significantly longer training sessions were carried out, given that the model needed to learn from scratch.

4.5 Model validation

The selected models were evaluated with the validation dataset, and the effectiveness of the model and the improvements provided by the hyperparameter selection, resulting from the large number of experiments carried out, were verified.

As can be seen in Table 4.4, the model with the best *performance* was the RGB color image model, which achieved 75% for *Precision*, $F_{1/2}$ -score of 0.693, 0.574 mAP and *Accuracy* of 44.9%. The false color model also offers good results, with 70% *Precision*, 40.2% *Accuracy*, $F_{1/2}$ -score of 0.643 and mAP of 0.515.

TABLE 4.4. Model validation results.

Metric	RGB ₈₆	FC ₆₈	Ratio
Inference time (<i>ms</i>)	11.395	12.528	-9.00%
<i>Accuracy</i>	44.90%	40.20%	+11.70%
<i>Precision</i>	75%	70%	+7.10%
mAP	0.574	0.515	+11.50%
$F_{1/2}$ -score	0.693	0.643	+7.80%
<i>G-measure</i>	0.630	0.583	+8.10%

Although initially, default parameters for confidence and IoU ($\text{conf}=0.25$ and $\text{iou}=0.6$) were used, it was later determined that better results could be achieved by fine-tuning these hyperparameters: for the RGB model, $\text{conf}=0.262$ and $\text{iou}=0.327$; and for the false color model, $\text{conf}=0.290$ and $\text{iou}=0.489$.

In this way, the model is able to offer a better count by not focusing on the correct localization of each *bounding box*, but rather on the effective detection of plants. The confusion matrix for each version can be seen in Figure 4.3.

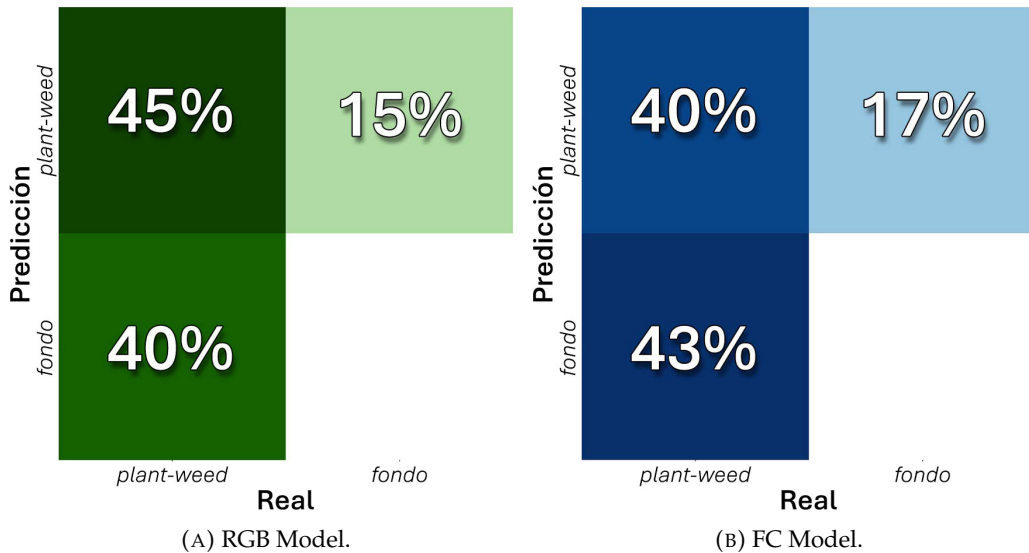


FIGURE 4.3. Confusion matrix for the validation set.

4.5.1 Impact of IoU and confidence

As explained above, due to the very nature of the challenge faced, it was a priority to have a reliable figure of detected plants without biasing the measurement upwards. Figure 4.4 shows the evolution of precision for each confidence level of the model.

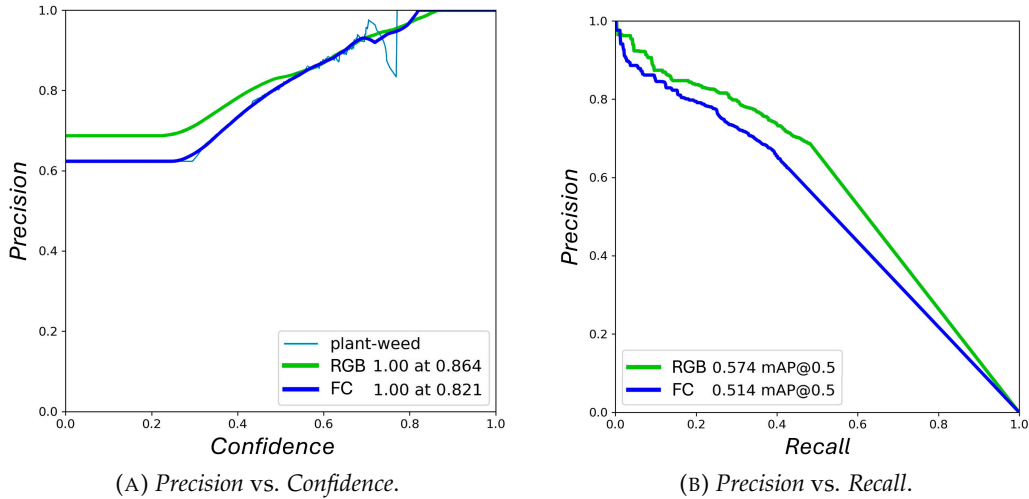


FIGURE 4.4. Precision curves as a function of confidence and recall.

By slightly increasing the confidence threshold (`conf`), the security required from the model to consider the detection valid, the number of false positives was significantly reduced from 21.03% to 14.79% (a reduction of -29.67%), for the RGB model; and from 27.16% to 17.20% (-36.67%), for the false color model (ExG+2-PCA+Burn).

Complementarily, by reducing the intersection over union percentage, the model was allowed to establish an optimal balance between detection sensitivity (*recall*) and their precision (*accuracy*), which is crucial for reducing false positives and achieving a count closer to the actual number of objects.

With these adjustments, mAP was slightly sacrificed (-0.52%) and *Accuracy* a bit more (-3.65%), to achieve a considerable improvement in *Precision* (+8.85%) and *F_{1/2}-score* (+3.90%). For the false color model, all representative metrics were increased (with an astonishing +17.45% for *Precision*) without sacrificing *Accuracy*.

4.6 Model testing

Finally, the model's performance was compared against the testing set. It can be seen in Table 4.5 that the results were similar to those achieved for the validation set.

Figure 4.5 shows that the choice of *F_{1/2}-score* as a design metric allowed the development of models that not only generalize results, but do so without increasing the number of false positives (FP).

TABLE 4.5. Model testing results.

Metric	RGB ₈₆	FC ₆₈	Ratio
Accuracy	42.1%	37.6%	+12%
Precision	77.8%	75.3%	+3.7%
mAP	0.563	0.476	+18.3%
F _{1/2} -score	0.691	0.654	+5.7%
G-measure	0.610	0.568	+7.4%

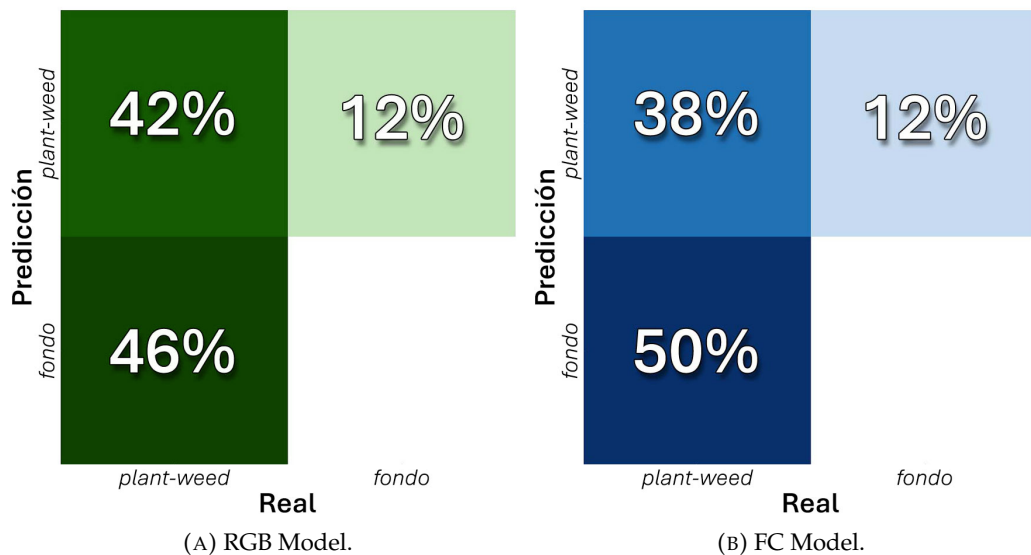


FIGURE 4.5. Confusion matrix for the testing set.

4.6.1 Impact of plant development on detection

The evaluated models were also tested exclusively with subsets of images at the same plant development stage (early and advanced) to observe performance in each scenario (Figure 4.6).

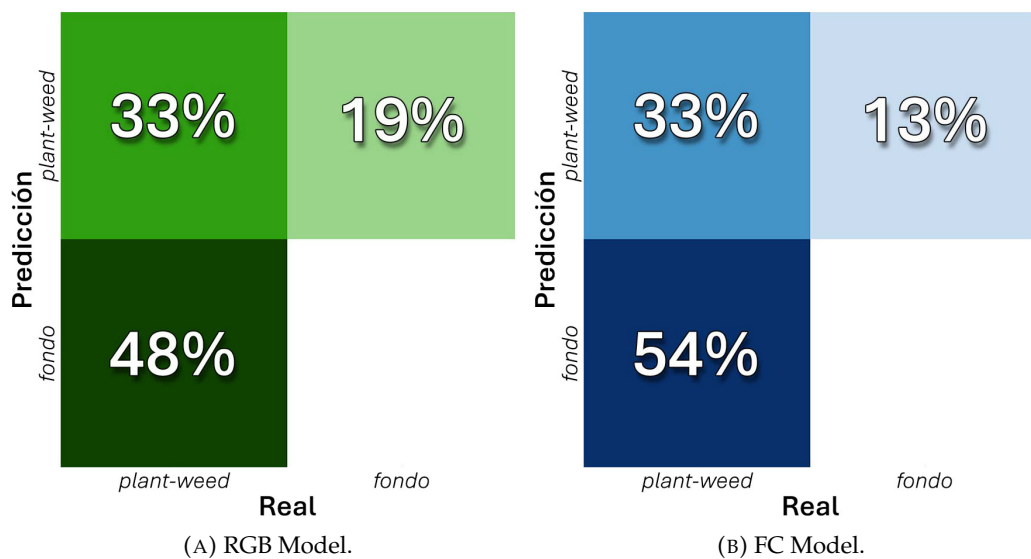


FIGURE 4.6. Confusion matrix for advanced development plant.

It can be seen that the RGB model offered significantly better results for the plant in early development. Table 4.6 shows that for more advanced stages, the model's detection capability progressively deteriorates and suffers a sharp drop in *Accuracy* caused by a notable increase in false negatives. False positives were kept constant, avoiding an upward bias in density measurement.

TABLE 4.6. RGB model response based on development and waterlogged terrain.

Metric	RGB ₈₆ (Reference)	Early development	Advanced development	Waterlogged lot
<i>Accuracy</i>	42.1%	47.4%	33.2%	48.9%
<i>Precision</i>	77.8%	70.9%	63.6%	86.6%
<i>Recall</i>	47.8%	58.8%	41.0%	52.9%
<i>mAP</i>	0.563	0.563	0.445	0.660
<i>F_{1/2}-score</i>	0.691	0.681	0.573	0.768
<i>G-measure</i>	0.610	0.646	0.510	0.677

However, it can be seen that by combining both subsets (reference), the model achieved a more balanced behavior in detections, which had a positive impact on precision and *F_{1/2}-score*, without harming *mAP* and averaging *Accuracy*.

On the other hand, a similar behavior was observed for the false color (FC) dataset, but with the particularity of performing much better for plants with advanced development, by managing to reduce false positives by half without sacrificing *Accuracy*. It can be seen in Table 4.7 that it achieved 72.6% *Precision*, which represents +12.7% improvement compared to the RGB model and +7.5% for *F_{1/2}-score*.

This shows that the false color model proposed here represents a good *trade-off* between computation time and measurement precision for plants in advanced development stages.

TABLE 4.7. FC model response based on development.

Metric	FC ₆₈ (Reference)	Early development	Advanced development
<i>Accuracy</i>	37.6%	42.7%	33.4%
<i>Precision</i>	75.3%	71.7%	72.6%
<i>Recall</i>	42.9%	51.3%	38.2%
<i>mAP</i>	0.528	0.539	0.487
<i>F_{1/2}-score</i>	0.654	0.664	0.616
<i>G-measure</i>	0.568	0.606	0.527

Finally, an additional testing trial was carried out with the RGB model for images of a waterlogged lot. The model was not trained with data with these characteristics, so it represents a good parameter of the variability that the model could face in productive situations. As can be seen in Table 4.6, the *performance* was even better than for crops under normal conditions. The model offered 86.6% precision (+22%), 48.9% *Accuracy*, with 0.66 *mAP* (+17%) and *F_{1/2}-score* 0.768. This is probably due to the fact that, with the terrain flooded, it is easier to differentiate the plant from the background than in normal conditions. An illustrative example can be seen in Figure 4.7.

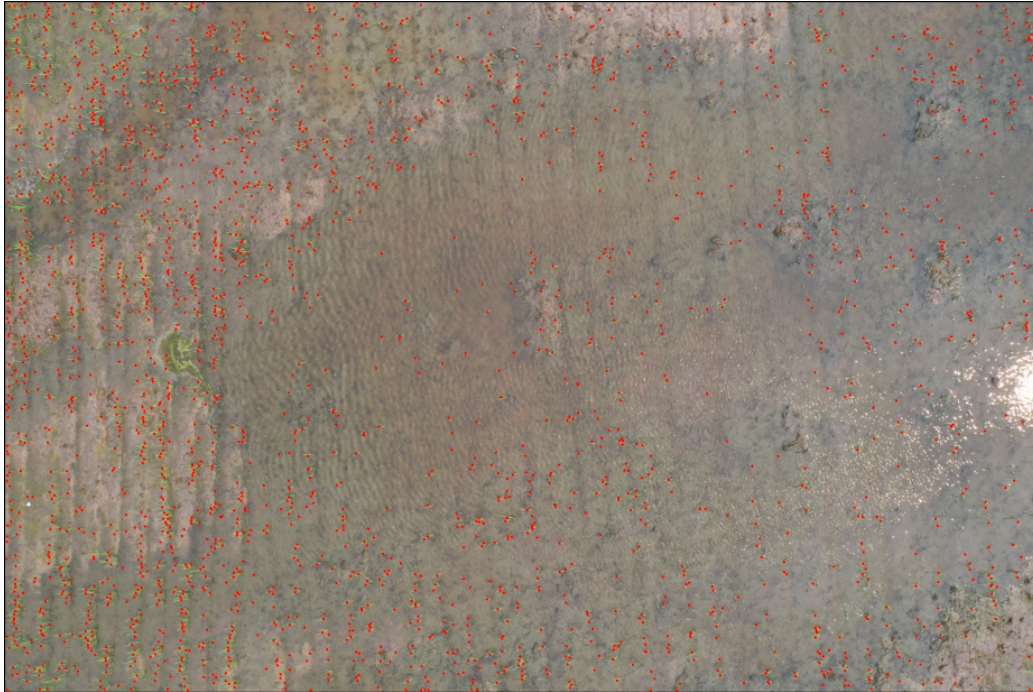


FIGURE 4.7. Example of inference for a waterlogged lot.

4.7 Performance in production

35 additional images were selected to emulate the model's performance under production conditions. All of these were data that had not been previously seen by the model, as they were not part of the training or validation sets.

Tests were carried out for different plant development stages, flight heights, and various problematic situations (waterlogging, presence of tracks, and blurred images). The impact of the IoU and *Confidence* configuration on the predictions made by the model was also evaluated.

These tests were performed with the inference *pipeline* (Figure 3.1) and each image was sent for plant counting. The measurement was contrasted with data taken in the field (ring count) and the lot density measurement provided by the client (*ground truth*). The root mean square error (RMSE) was calculated to measure the difference between each measurement.

Upon a general visual inspection of the detections, their consistency was verified given the crop distribution and the correct discrimination of weeds. All correspond effectively to plants (no detections of stubble or other strange objects as false positives were observed).

It is important to note that the error measure is merely indicative, since:

1. To truly measure the effectiveness of the proposed method under production conditions, it would be necessary to carry out experiments designed for this purpose: take all necessary samples from the same lot and perform batch processing.
2. The ring count, although precise, is not representative of the actual lot density, as it is a single sample with an extremely small area.

3. The reference measurement provided by the client has a large margin of error and there is no certainty of the statistical representativeness of said measurement (the total number of samples taken during the traditional measurement has not been reported).

At the time of writing the report, there was no data that met these requirements, so it was not possible to perform representative tests. This is why it was decided simply to verify the system's response to diverse conditions, without evaluating the precision of the statistical measurement *per se*.

4.7.1 Characterization of the productive system

The traditional distinction between extensive and intensive agricultural systems becomes blurred in the face of large-scale production models that incorporate a high level of technology and intensive management. Extensive systems are traditionally characterized by covering large areas with low inputs per hectare and less mechanization, with the aim of maximizing total production rather than unit yield.

However, the client's production model contrasts with this categorization. Despite operating over a large area, typical of an extensive system, its agronomic management is fundamentally intensive. This character is manifested in the advanced use of technology and the constant search for high productivity. The main objective is to maximize efficiency and yield per hectare.

Therefore, the combination of the vast scale with high-intensity technological and input agricultural practices forces these models to be considered predominantly intensive, despite their spatial dimension.

The consulted *papers* sought to solve plant counting in experimental plots with smaller areas or in crops with significantly lower densities and quantities of plants. It is essential to highlight that the tool developed here, being a productive tool that seeks to cover large extensions of cultivation (200 ha lots) with high plant density (200 plants/ha), had to face significantly more complex challenges than those represented by experimental plots. This constitutes a differential point of the solution provided in this work compared to the state of the art.

Critique of the traditional method

The traditional counting method involves walking or riding a motorcycle through the lot and taking at least 10 samples with a 0.25 m² ring. According to the client, the ring count offers an estimated precision of ± 10 plants over the ideal density of 200 plants/m². This translates into a margin of error of $\pm 5\%$. As detailed in appendix B, to reach a confidence level of 90%, a minimum of 17 rings would be needed, while for 95% confidence, 24 rings would be required. According to the information, due to resource limitations and the number of hectares to cover, they currently perform the measurement with 10 samples, which offers a confidence level of 80%.

As calculated in appendix B, to perform an equivalent measurement with the proposed model, given the precision achieved, it would be advisable to capture at least 55 samples from the same lot to achieve a measurement with a margin of error of $\pm 5\%$ and 80% confidence.

4.7.2 Impact of plant development on *performance*

It is verified that the best results are obtained for plants in the early (209) and advanced (503) development stages, where detections have a unique relationship with the plants present in the image.

It was verified that for late stages of development (greater than 3 weeks) the results offered by the model are not useful, as it becomes unable to recognize individual plants (due to tillering). It only recognizes plant clusters, which makes density measurement impossible.

4.7.3 Reflection on confidence and IoU threshold

It is possible to make a comparison between the "optimized" measurement (with customized IoU and *confidence* thresholds) and the model's default detections. By visually inspecting Figure 4.8 and comparing the detections for each case, it can be observed that by adjusting the thresholds (red dots), false positives can be better filtered by avoiding counting the same plant multiple times. In the standard detection (blue dots), a large agglomeration of detections can be seen in regions of the image where there are few plants.

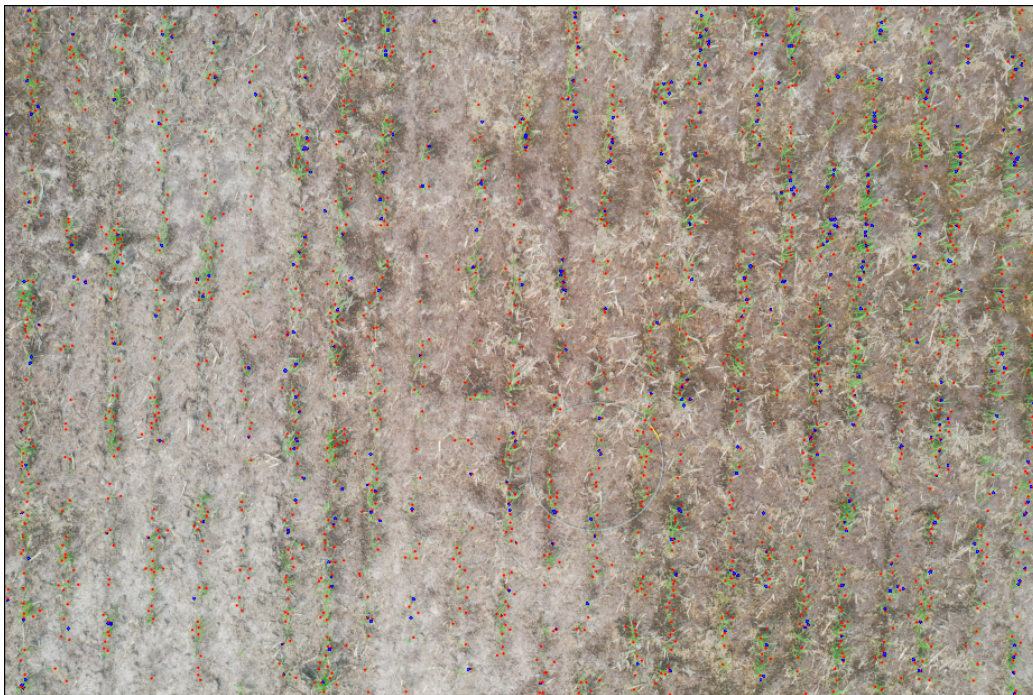


FIGURE 4.8. Impact of IoU and *Confidence* on inference.

4.7.4 Problematic cases

Inferences made for common problematic situations offered results equivalent to those achieved in normal situations, demonstrating that the design strategies implemented for this purpose were effective.

In particular, the model was able to perform counting in waterlogged lots, stubble, or machinery and operator tracks, without affecting the measurement, as seen in Figure 4.7.

Additionally, it responded adequately to images with a high level of blur, generally caused by the presence of strong winds during capture, which prevent the drone from stabilizing enough to achieve optimal focus.

4.7.5 Impact of flight height on performance

Finally, counts were performed for the same point of the lot photographed at different heights at 0.5-meter intervals to evaluate the impact of flight height on performance. The detailed results can be seen in Table A.2.

Although the model was trained with images captured at an altitude of 3.5 meters, it is possible to perform inference in a range of heights without this generating a negative impact on the measurement.

A useful range for inference between heights of 1.5 to 6.5 m was verified, with optimal values around 2 and 4.5 m to achieve a good balance between detail and sampled area. From 7 m height, a clear deterioration in the number of detections is observed.

The impact of height is more noticeable for plants in early development (being smaller). For plants with advanced development, it is possible to slightly increase the flight height and approach the limits of the aforementioned useful range without losing precision.

4.8 Estimation of necessary resources

According to the calculation made in appendix B, it would be necessary to take at least 30 samples (images from the same lot) to perform a statistically representative measurement of a standard 200 ha lot.

Given the current precision achieved by the best model, it is recommended to work with 55 images to achieve a measurement with confidence levels and margin of error equivalent to those obtained by the client with traditional methods.

By measuring the processing times for each of the stages, an estimation of the resources needed to process a *batch* of n samples from the same lot can be made.

As can be seen in Table 4.8, capturing 30 images would require ~22 min for drone flight and ~9 min for image processing to measure plant density, i.e., a little over half an hour to complete the task. This represents a decrease in the time required to perform the task of up to 87.5% compared to traditional methods.

The flight autonomy for the DJI Mavic 2 Pro is 30 min, but it is advisable not to extend the flight beyond 25 min so that the drone can return to base and make a safe landing. Within this time window, it is possible to increase the number of samples without needing to change the battery (estimated to take 7 min).

Although increasing the number of photos extends the flight time (because the drone needs to stabilize before taking the photo, which decreases the average flight speed), this does not represent a significant increase.

From Table 4.8, it can be deduced that if the number of samples were increased from 30 to 55 (+83

Thanks to this non-linear behavior, the method is easily scalable, which allows for a good balance between the time dedicated to the task and the desired precision in the measurement.

On the other hand, if the alternative model for false color images is to be used, the computation time doubles due to the multiple image transformation stages required. As seen in Table 4.8, the total time for 30 images would in this case amount to 41 min (result of $31.2 + 10.4$ min).

Furthermore, it is advisable to perform inference locally, as cloud processing was what most increased processing time ($\sim 6x$). When computing in the cloud (+45 min) or requiring file transfer via the Internet (+6 min), the duration of the counting task can amount to more than 1 hour for 30 images or up to 2 ½ hours for 55 photographs, representing a 6x increase. The duration of each process can be seen in detail in Table A.1 of the appendix.

Regarding storage usage, at least 150 MB are needed to store 30 images (5 MB each) and an additional temporary space of +11.4 MB/image during processing. That is, a total disk space of 492 MB. For 50 samples, 250 MB would be needed for images and 820 MB for temporary files. Both are reasonable values for current standard equipment capacities.

4.8.1 Contribution to productivity

It can therefore be concluded that it is feasible to increase the number of images in the same batch according to the desired margin of error for the measurement and the resources available to perform the task.

It is reasonable to assume a total time of less than 1 hour to complete the counting task (drone flight + computation). This represents an efficiency improvement of at least 5x compared to counting with traditional methods.

The proposed solution for plant counting in rice crops using an ML *pipeline* provides the following solutions:

- Reduces the need for trained personnel, as drone flight can be programmed in advance.
- Eliminates the human factor, which could have introduced bias into the measurement.

TABLE 4.8. Estimation of time for the counting task.

Process	Lot 30 images	Lot 55 images	Increase
Drone flight	22 min	40 min	1.82x
Transfer ($\downarrow 300 \mid \uparrow 50$ Mbps) *	4.5 ± 1.5 min	8.3 ± 2.8 min	1.83x
Inference <i>pipeline</i>	9.2 min	15.2 min	1.65x
Local processing (RGB)	31.2 min	55.2 min	1.77x
Additional ExG+2-PCA	+10.4 min	+19 min	1.83x
Additional cloud * (computation + transfer)	+49.5 min	+89 min	1.80x

- Streamlines the field task, by limiting it to drone flight, without the need to enter the lot.
- Allows for easy scalability with the increase in hectares produced.

Chapter 5

Conclusions

This chapter details the conclusions drawn from the results of the present work on the problem of plant counting in rice crops.

The findings validate the viability of the proposed method and achieve an effective balance between precision in density measurement and resource optimization.

5.1 General conclusions

Despite the complexity of the problem faced, the results obtained are promising. The solution developed in this work facilitates plant counting in large plots and achieves a good balance between the precision of plant density measurement and the use of human and material resources.

The model responds satisfactorily to the conditions for which it was trained (plants in early or intermediate development stages). Furthermore, it is robust to the variability of input conditions. It offers adequate *performance* in variable lighting conditions, with flooded plots or in the presence of foreign objects (such as stones, snails, or stubble) and traces of operators and machinery. It is even capable of detecting plants with blurred images.

The combined application of the proposed models provides a versatile, robust, and efficient solution for different development stages and crops with variable characteristics. The proposed solution works under practical time and hardware requirements, which allow increasing productivity by more than 500%.

Although there is still room for improvement of the model, mainly if the dataset is nourished with a greater number and diversity of data, the viability of the proposed method has been proven and the path has been opened for future lines of research for superior solutions.

In addition, within the framework of this work, innovative techniques were developed for processing crop images through the combination of vegetation indices (ExG), dimensional reduction (PCA), and image processing techniques (combination, fusion, CLAHE, and masks). As a result of this exploration, a set of derived data was created that makes it easier to differentiate plants from the background.

5.2 Next steps

In the future, it would be interesting to be able to work with only 30 samples to carry out all image capture during a single drone flight (without needing to change the battery). In this way, the total task time could be reduced to less than 30 minutes.

Although the proposed alternative model that works with false-color images did not manage to outperform the RGB image model, the results obtained with this processing are promising and suggest that, if the development of this model continues, the current results could be greatly improved.

Furthermore, it was observed that the application of PCA allowed optimal separation of the background (first component) from the plants (second component). It is to be assumed that this technique can be applied in different ways than those explored during this work.

Some exploration routes for future work include:

- Continue the analysis of the false-color dataset (PCA + ExG + *Burn Blend*) to achieve better results.
- Enrich the dataset with new images that meet the ideal characteristics defined in the present work.
- Incorporate into the training set a greater number of examples of problematic situations (such as stubble, flooding, plots with a large presence of weeds), different environmental situations (crop characteristics), or more images of soil without plants.
- To improve the quality of labeling, it is proposed:
 - Work exclusively in the early stage of plant development.
 - Review critical cases with a specialist.
 - Evaluate the use of false-color images during labeling.
- Investigate advances in the state of the art and implement other architectures.
- Explore the incorporation of semantic segmentation models or hybrid architectures that combine detection and segmentation.
- Experiment with larger scale YOLOv8 models (*Large* or *Extra Large*).
- Evaluate the use of `cluster` plant labels (overlapping plants) to develop a parallel model capable of detecting potentially problematic points for counting.
- Analyze the incorporation of some type of heuristic for low-confidence labels (`occluded`) or implement special processing techniques for these problematic objects.
- Conduct productive field tests with new drone images to evaluate the model's performance in productive environments.

- Capture the multiple samples necessary for one or more plots, and thus experimentally verify the performance of the inference *pipeline* during batch processing.

Appendix A

Inference results

Table A.1 details the stages of the inference *pipeline* and the duration of each task during the computation of an individual sample, and with batches of 30 and 55 images.

TABLE A.1. Computation time for batch processing.

Stage	Task	Times per image	Batch 30 images	Batch 55 images
Pre-process	ExG+2-PCA **	20.8 sec	624 sec	1144 sec
	<i>Tiles splitter</i>	500 ms	15 sec	27.5 sec
	CLAHE	486 ms	14.58 sec	26.73 sec
	<i>Upload *</i>	40 sec	1200 sec	2200 sec
YOLOv8 Model	Initialization	60 sec/batch	60 sec	60 sec
	<i>Extra cloud *</i>	70 sec/batch	70 sec	70 sec
	Inference	2341.6 ms	60 sec	110 sec
	<i>Download *</i>	47 sec	1410 sec	2585 sec
Post-process	<i>Tiles un-splitter</i>	4 sec	120 sec	220 sec
	<i>Line detection</i>	8.5 sec	255 sec	467.5 sec
Local processing		15.8 sec	9.2 min	15.2 min
Additional cloud * (computation + transfer)			+45 min (5.9x)	+81 min (6.3x)
Additional ExG+2-PCA **			+10.4 min (2.1x)	+19 min (2.3x)

Table A.2 shows the results provided by the model for images captured in the same sector of the lot at different heights (a measurement with a single sample from the lot). It can be observed that the RMSE between the density calculated by the model and the measurement—for the entire lot—reported by the client (*ground truth, GT*), error values dependent on the flight height at which the images were taken are obtained.

If the density measured with the measuring ring ($\frac{1}{4}$ m² coverage) for the same sector of the lot is taken as a comparison point and contrasted with the GT for the entire lot, an RMSE of 36 is obtained. This value is reasonable given the variability of crop density within the same lot and is statistically compensated by repeating the measurement for multiple equally distributed rings.

In this specific example, the minimum error value was obtained for a height of 2.5 m. Experimentally, a useful range between 2 and 4 meters was determined to obtain a good balance between the covered area and the resolution per pixel.

If the RMSE of the ring is compared with that of the model's measurement, it can be seen that in this example, for heights close to 2.5 m, errors lower than those of the ring measurement were obtained.

It is important to note that this test was performed for a single sample of the lot due to lack of data availability. As explained in the conclusions of the work, to perform these measurements accurately, it would be necessary to capture the number of samples required by the model in a productive environment and, in addition, to carry out an exhaustive analysis of the margin of error of the traditional method.

TABLE A.2. Detections for the same sector at different heights.

Flight height (meters)	Area covered (m ² /img)	Detections (plants/img)	Calculated density (plants/m ²)	RMSE Error (Model vs. GT)	Useful measurement range
1	1.0	392	385.6	205.6	✗
1 ½	2.3	636	245.3	65.3	✓
2	4.4	941	213.3	33.3	✓
2 ½	6.8	1129	167.2	12.8	✓
3	9.9	1324	133.4	46.6	✓
3 ½	13.0	1439	111.0	69.0	✓
4	16.8	1523	90.7	89.3	✗
4 ½	21.0	1612	76.7	103.3	✗
5	25.4	1617	63.6	116.4	✗
5 ½	35.2	1533	43.6	136.4	✗
6	41.8	1571	37.6	142.4	✗
6 ½	46.4	1514	32.6	147.4	✗
7	55.0	1395	25.4	154.6	✗
7 ½	64.0	1276	19.9	160.1	✗
8	70.6	1145	16.2	163.8	✗
8 ½	78.2	1043	13.3	166.7	✗
9	90.5	887	9.8	170.2	✗
9 ½	101.7	817	8.0	172.0	✗
10	110.3	784	7.1	172.9	✗

Appendix B

Number of samples and measurement precision

Statistical development

Parameters reported by the client

- Number of samples: 10 rings
- Sampling area: $0,25\text{ m}^2$
- Lot area: $\sim 200\text{ ha} = 200 \cdot 10\,000\text{ m}^2 = 2\,000\,000\text{ m}^2$
- Variability: density varies between 150 to 250 plants/ m^2 for different sectors of the lot.
- Precision (Margin of Error): $\pm 10/200 = \pm 5\%$
- Confidence Level: 90% and 95% levels will be considered.

B.1 Cluster sampling

In single-stage cluster sampling, where all elements within selected clusters are sampled, the population is divided into M clusters. The formula for the number of clusters (m) to select is different, and precision depends on variability between clusters.

The formula for the number of samples (m) to take to estimate the population mean with an absolute error E is:

$$m = \frac{Z^2 \cdot S_c^2}{E^2 + \frac{Z^2}{M} \cdot S_c^2}$$

Where:

- M is the total number of *clusters* in the population (samples taken),
- S_c^2 is the estimated variance of the average *clusters*,
- E is the desired absolute margin of error,
- Z is the weight for the desired confidence level.

Sampling unit (*cluster*) and number of clusters

Given that each sample is taken with a $0,25\text{ m}^2$ ring, the number of clusters will be:

$$M = \frac{\text{Total Area}}{\text{Cluster Area}} = \frac{2\,000\,000\text{ m}^2}{0,25\text{ m}^2} = 8\,000\,000\text{ clusters}$$

Estimated variance

Based on the observed range (150 to 250 plants/ m^2) in density per m^2 in different sectors of the lot,

$$\Delta x = x_{\max} - x_{\min} = 250 - 150 = 100\text{ plants/m}^2$$

and assuming that it represents approximately 4 standard deviations, the standard deviation of the *clusters* ($S_{\bar{c}}$) will be:

$$S_{\bar{c}} \approx \frac{\Delta x}{4} = 25\text{ plants/m}^2$$

Number of samples

Knowing that the absolute error $E = 10\text{ plants/m}^2$.

Case 1

For 90% Confidence, $Z = 1,645$:

$$m = \frac{Z^2 \cdot S_{\bar{c}}^2}{E^2 + \frac{Z^2}{M} \cdot S_{\bar{c}}^2}$$

$$m_{90} = \frac{(1,645)^2 \cdot (25)^2}{10^2 + \frac{(1,645)^2}{8 \cdot 10^6} \cdot (25)^2} = \frac{2,706025 \cdot 625}{100 + \frac{2,706025}{8 \cdot 10^6} \cdot 625} = \frac{1691,265625}{100,000211408} \approx 16,91$$

Rounding to the nearest integer, the number of samples needed to obtain a measurement with 90% confidence is 17 rings.

Case 2

For a 95% confidence level, $Z = 1,96$:

$$m_{95} = \frac{(1,96)^2 \cdot (25)^2}{10^2 + \frac{(1,96)^2}{8 \cdot 10^6} \cdot (25)^2} = \frac{3,8416 \cdot 625}{100 + \frac{3,8416}{8 \cdot 10^6} \cdot 625} = \frac{2401}{100,00030125} \approx 24,0099$$

That is, for a measurement with 95% confidence, 24 rings will be necessary.

Case 3

Taking the number of rings reported by the client, we can also calculate the confidence level of their measurement.

Solving for Z :

$$\begin{aligned}
 m &= \frac{Z^2 \cdot S_c^2}{E^2 + \frac{Z^2}{M} \cdot S_c^2} \\
 m \left(E^2 + \frac{Z^2}{M} \cdot S_c^2 \right) &= Z^2 \cdot S_c^2 \\
 mE^2 + m \frac{Z^2}{M} \cdot S_c^2 &= Z^2 \cdot S_c^2 \\
 mE^2 &= Z^2 \cdot S_c^2 - m \frac{Z^2}{M} \cdot S_c^2 \\
 mE^2 &= \left(1 - \frac{m}{M} \right) Z^2 \cdot S_c^2 \\
 Z^2 &= \frac{mE^2}{S_c^2 \left(1 - \frac{m}{M} \right)}
 \end{aligned}$$

Substituting the values $E = 10$, $S_c = 25$, $M = 8 \cdot 10^6$:

$$\begin{aligned}
 Z_{m=10}^2 &= \frac{10 \cdot 10^2}{25^2 \left(1 - \frac{10}{8 \cdot 10^6} \right)} = \frac{10^3}{625 \left(1 - \frac{1}{8 \cdot 10^5} \right)} = \frac{1,6}{0,99999875} \approx 1,600002 \\
 \Rightarrow Z_{m=10} &= \sqrt{1,600002} \approx 1,26491185
 \end{aligned}$$

Consulting the value in a standard normal distribution table, we can see that the total accumulated probability for the value $Z_{m=10}$ corresponds to $\approx 0,89617$. Given that the normal distribution is symmetrical, the accumulated probability from the mean is:

$$0,89617 - 0,5 = 0,39617$$

Then, the area for both tails of the distribution —outside the confidence interval— is:

$$2 \times (0,5 - 0,39617) = 2 \times 0,1028 = 0,2056$$

In conclusion, the Confidence Level —area within the interval— for 10 samples will be:

$$1 - 0,2056 = 0,7944 \approx 80\%$$

Relationship between *Accuracy* and sample size

The solution proposed in the thesis work consists of using a Computer Vision object detection model that:

1. Identifies the location of the plants.
2. Counts the total number of detections.
3. Calculates plant density, knowing the area covered by the aerial image.

The objective of this appendix is, with the model's metrics (*accuracy*, *precision* and *recall*), to try to estimate the number of necessary samples (drone photos) to perform a measurement with a certain confidence interval.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad Precision = \frac{TP}{TP + FP} \quad Recall = \frac{TP}{TP + FN}$$

Where:

- TP are True Positives,
- FP are False Positives,
- TN are True Negatives,
- FN are False Negatives.

Relationship between Accuracy and estimated proportion

Within the context of *Machine Learning*, a statistical approach can be adopted by considering the n inferences made by the model for the same lot as a random sample of predictions, where each prediction can be correct or incorrect.

This aligns with the concept of Bernoulli trials.

Where:

$$P(X = x) = p^x(1 - p)^{1-x}$$

- $P(X = x)$ is the probability that the random variable X takes the value x ,
- x is the outcome of the attempt, which can only be 0 (failure) or 1 (success),
- p is the probability of success in a single attempt (value between 0 and 1),
- $1 - p$ is the probability of failure.

It can be broken down into two cases:

$$P(X = 1) = p \quad P(X = 0) = 1 - p$$

With distribution function:

$$F(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 - p & \text{if } 0 \leq x < 1 \\ 1 & \text{if } x \geq 1 \end{cases}$$

And parameters:

$$E[X] = p \quad \text{Var}[X] = p(1 - p)$$

Starting from the previous premise that the crop presents a normal distribution, it is possible to make an approximation from the binomial distribution, a principle based on De Moivre-Laplace's Theorem.

Therefore, each inference for the same lot can be modeled as a Bernoulli trial with a probability of success p (hit).

As a consequence, the total number of correct predictions will follow a binomial distribution $B(n, p)$.

Wald Interval (normal approximation)

As long as the premises of De Moivre-Laplace's theorem are met and n is sufficiently large, the binomial distribution can be approximated by a normal distribution with mean $\mu = np$ and variance $\sigma^2 = np(1 - p)$.

The Wald Interval formula uses this approximation to construct a confidence interval for the population proportion p based on the sample proportion of correct predictions.

This can be defined as $\hat{p} = \frac{\text{hits}}{n}$ with approximately normal distribution with mean p and standard error $SE = \sqrt{\frac{p(1-p)}{n}}$.

To construct the $1 - \alpha$ confidence interval, we use the expression $Z_{\alpha/2}$ such that $P(-Z_{\alpha/2} \leq Z \leq Z_{\alpha/2}) = 1 - \alpha$, where Z is the standard normal variable.

Then, the interval can be defined as:

$$P\left(\hat{p} - Z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}} \leq p \leq \hat{p} + Z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}\right) = 1 - \alpha$$

Within these assumptions, it holds that:

$$\forall n\hat{p} \geq 5 \quad \wedge \quad n(1 - \hat{p}) \geq 5;$$

$$p \approx \hat{p} \pm Z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}} = \hat{p} \pm ME$$

Where:

- $\hat{p}$ is the sample proportion (estimator of the population proportion p),
- $Z_{\alpha/2}$ is the critical value of the normal distribution,
- ME represents the distance from the sample estimate to the endpoints of the confidence interval E .

From this, it is deduced that the sample size must always be $n \geq 10$, since for the estimator to work there must be at least 5 successes and 5 failures:

$$\begin{aligned} n\hat{p} &\geq 5 \\ n \cdot \left(\frac{x}{n}\right) &\geq 5 \\ x &\geq 5 \end{aligned}$$

$$\begin{aligned} n(1 - \hat{p}) &\geq 5 \\ n \cdot \left(1 - \frac{x}{n}\right) &\geq 5 \\ n - x &\geq 5 \end{aligned}$$

Within this framework, the model's *Accuracy* metric, calculated for a representative evaluation dataset, can serve as a point estimate $\hat{p}$ of the true proportion of correct predictions for the lot population.

$$\hat{p} = \frac{\text{hits}}{n} \equiv \text{Accuracy}$$

As long as a sufficiently large number n of independent Bernoulli trials are performed, each with probability of success p , where the distribution of the number of successes approximates a normal distribution.

It is important to note that for this approximation to work, it must be an "imperfect" model.

If the model gets all predictions right, the probability of success will be $p = 1$. Otherwise, if it never gets it right, the probability will be $p = 0$. In both cases, the necessary conditions for applying the Wald Interval are no longer met: $n\hat{p} \geq 5$ (successes) and $n(1 - \hat{p}) \geq 5$ (failures).

B.2 Impact of sample size on Accuracy

The estimated proportion $p(1 - p)$ represents the variance of a Bernoulli random variable with probability of success p .

If p is the true proportion of a characteristic in a population, then $p(1 - p)$ will represent the variance of an individual Bernoulli observation.

In statistics, the variance of a sample proportion $\hat{p}$ (estimated from a population with true proportion p) is theoretically approximated to $p(1 - p)/n$, where n is the number of samples.

In the context of *Machine Learning*, although *Accuracy* can be considered an estimate of the true probability that the model classifies correctly, $\hat{p} = \text{Accuracy}$, it is not common to directly use the expression $\text{Accuracy}(1 - \text{Accuracy})$ as a performance metric, since this reaches its minimum for values of $\text{Accuracy} = 0$ or $\text{Accuracy} = 1$, and its maximum for $\text{Accuracy} = 0,5$.

For this reason, it does not adequately relate to the idea of metric improvement. Instead, it functions as a measure of the intrinsic variability inherent to this metric calculated on a finite sample.

Instead, it is more appropriate to use the Precision metric as an estimated proportion of success, as it effectively describes the proportion between hits and errors in detection. Another option, when no prior information is available, is to use a value of $p = 0,5$ as a conservative estimate.

Sample size needed for inference

The formula to calculate the necessary sample size (n) to estimate a population proportion (p) with a margin of error (E) and a confidence level (Z) is:

$$n = \frac{Z^2 \cdot p \cdot (1 - p)}{E^2}$$

Where:

- n is the necessary sample size,
- Z is the critical value corresponding to the desired confidence level (e.g. $Z = 1,96$ for 95% confidence, $Z \approx 1,28$ for 80% confidence),
- p is the estimated proportion of success (in this case, the model's Precision),
- E is the allowed margin of error (in this case, 0,05 for $\pm 5\%$).

Then, knowing that the area covered by the aerial image is $\sim 10\text{ m}^2$, an adjustment for finite populations can be made:

$$n_{\text{adjusted}} = \frac{n}{1 + \frac{n-1}{M}}$$

Where M is the total number of possible *clusters* (photos) in the lot:

$$M = \frac{\text{Total lot area}}{\text{Area covered by photo}} = \frac{2\,000\,000\text{ m}^2}{10\text{ m}^2/\text{photo}} = 200\,000\text{ clusters (photos)}$$

Case 1

From the tests performed with the model, the Precision metric is available, so we will use this parameter as the p value:

$$p = \text{Precision} = 0,78 \quad (1 - p) = 1 - 0,78 = 0,22$$

For the calculated confidence level (of $\approx 80\%$), $Z = 0,896$ is used:

$$\begin{aligned} n_{80\% \text{ approx}} &= \frac{Z^2 \cdot p \cdot (1 - p)}{E^2} = \frac{(0,896)^2 \cdot 0,78 \cdot 0,22}{0,05^2} = \\ &\approx \frac{0,802816 \cdot 0,1716}{0,0025} \approx \frac{0,137763}{0,0025} \approx 55,10 \end{aligned}$$

$$\begin{aligned} n_{\text{adjusted}} &= \frac{n}{1 + \frac{n-1}{M}} = \\ &= \frac{55,10}{1 + \frac{55,10-1}{200\,000}} = \frac{55,10}{1,00027053} \approx 55,085 \end{aligned}$$

Then, rounding to the nearest integer, it can be concluded that the number of samples needed to obtain a statistically representative measurement, given the precision of the current model, is 55 photos for 80% confidence with a 5% error (acceptable values for the client).

Case 2

Conversely, we can calculate the precision p that the model must achieve if it is desired to work with a restriction of the number of samples $n = 30$, maintaining

$E = 0,05$ and $Z = 0,896$ (for $\sim 80\%$ confidence):

$$n = \frac{Z^2 \cdot p \cdot (1 - p)}{E^2}$$

$$nE^2 = Z^2 \cdot p \cdot (1 - p)$$

$$p \cdot (1 - p) = \frac{nE^2}{Z^2}$$

Substituting the values, for $n = 30$:

$$p_{30} \cdot (1 - p_{30}) = \frac{30 \cdot (0,05)^2}{(0,896)^2} = \frac{30 \cdot 0,0025}{0,802816} \approx \frac{0,075}{0,802816} \approx 0,09342116$$

$$p_{30}^2 - p_{30} + 0,09342116 = 0$$

Solving the quadratic equation of the form $ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$, where $a = 1, b = -1, c = 0,09342116$:

$$\begin{aligned} p_{30} &= \frac{1 \pm \sqrt{(-1)^2 - 4 \cdot 1 \cdot 0,09342116}}{2 \cdot 1} = \frac{1 \pm \sqrt{1 - 0,37368464}}{2} = \\ &= \frac{1 \pm \sqrt{0,62631536}}{2} \approx \frac{1 \pm 0,79140088}{2} \end{aligned}$$

$$\begin{cases} p_{30A} = \frac{1+0,79140088}{2} \approx 0,89570044 \\ p_{30B} = \frac{1-0,79140088}{2} \approx 0,10429956 \end{cases}$$

We can see that, of the values that solve the quadratic equation, one (p_{30B}) makes no practical sense if the aim is to improve the model's precision (unless the current model is worse than random and a very low but specific precision is desired). Generally, the highest precision is sought.

$$Precision_{30} \approx 89,57$$

In conclusion, to be able to work with only 30 images and similar client requirements to the current ones (Confidence Level $\sim 80\%$ with $Z = 0,896$ and Margin of Error 5%), the model should achieve a precision of $\sim 90\%$.

Appendix C

Drone flight protocol

Objective

Image capture in post-emergence rice crops at an early growth stage to develop artificial intelligence models that facilitate and streamline plant counting.

Requirements (Checklist)

- ☐ Include reflectance panel in the first image.
- ☐ Verify flight configuration.
- ☐ Adjust resolution and verify brightness.
- ☐ Verify favorable flight conditions (wind).
- ☐ At the end, verify that the images are in focus.

Flight Plans

Drone Models

DJI Mavic 2 Pro, DJI Mavic Air 2 or DJI Mavic 3T.

Flight #1 - Zenith Capture

- Flight height: 3 meters.
- Capture angle: 90° (zenithal).
- Number of stops per lot: 10.
- Image resolution: Maximum, 48MP (Air2 or 3T) or 20 MP (2Pro).
- Capture mode: Single shot.
- Measurement modality: Center-weighted average.
- Exposure program: Normal.
- File format: JPEG.

Flight #2 - Angled Capture

- Flight height: 3 meters.
- Capture angle: 60°.
- Number of stops per lot: 10.
- Image resolution: Maximum, 48MP (Air2 or 3T) or 20 MP (2Pro).
- Capture mode: Single shot.
- Measurement modality: Center-weighted average.
- Exposure program: Normal.
- File format: JPEG.

Appendix D

Hardware resources

For model training, hardware resources offered by Google Colab were used. In particular, the NVIDIA Tesla T4 graphics card was used, whose specifications can be consulted in table [D.1](#).

TABLE D.1. Specifications of the NVIDIA Tesla T4 GPU.

Characteristic	Value
Architecture	NVIDIA Turing
CUDA Cores	2,560
Tensor Cores	320
Memory	16 GB GDDR6
Memory Bandwidth	Up to 320 GB/s
Peak FP32 Performance	8.1 TFLOPS
Mixed Precision (FP16/FP32)	65 FP16 TFLOPS
INT8 Performance	130 INT8 TOPS
INT4 Performance	260 INT4 TOPS
Power Limit	70W
Form Factor	Single-slot, low-profile, 6.6-inch
Cooling	Passive, requires system airflow
System Interface	PCIe 3.0 x16

Bibliography

- [1] Adecoagro S.A. *1Q24 Earnings Release*. www.adecoagro.com. 2024. (Visited on 03/17/2025).
- [2] Adecoagro S.A. *Arroz*. Disponible: 2025-03-25. URL: <https://www.adecoagro.com/es/nuestros-negocios/arroz>.
- [3] H. García-Martínez et al. "Digital Count of Corn Plants Using Images Taken by Unmanned Aerial Vehicles and Cross Correlation of Templates". In: *Remote Sensing* 12.7 (2020), p. 1101. DOI: [10.3390/rs12071101](https://doi.org/10.3390/rs12071101).
- [4] J. Valente et al. "Automated crop plant counting from very high-resolution aerial imagery". In: *Precision Agriculture* 21.1 (2020), pp. 1366–1384. DOI: [10.1007/s11119-020-09725-3](https://doi.org/10.1007/s11119-020-09725-3).
- [5] S. Madec et al. "Ear density estimation from high resolution RGB imagery using deep learning technique". In: *Agricultural and Forest Meteorology* 264 (2019), pp. 225–234. DOI: [10.1016/j.agrformet.2018.10.013](https://doi.org/10.1016/j.agrformet.2018.10.013).
- [6] X. Xu et al. "Detection and counting of maize leaves based on two-stage deep learning with UAV-based RGB image". In: *Remote Sensing* 14.21 (2022), p. 5388. DOI: [10.3390/rs14215388](https://doi.org/10.3390/rs14215388).
- [7] A. Ammar, A. Koubaa, and B. Benjdira. "Deep-Learning-Based Automated Palm Tree Counting and Geolocation in Large Farms from Aerial Geotagged Images". In: *Agronomy* 11.8 (2021), p. 1458. DOI: [10.3390/agronomy11081458](https://doi.org/10.3390/agronomy11081458).
- [8] B. Wang et al. "Multiscale Maize Tassel Identification Based on Improved RetinaNet Model and UAV Images". In: *Remote Sensing* 15.10 (2023), p. 2530. DOI: [10.3390/rs15102530](https://doi.org/10.3390/rs15102530).
- [9] A. Karami, M. Crawford, and E. J. Delp. "Automatic Plant Counting and Location Based on a Few-Shot Learning Technique". In: *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 13.1 (2020), pp. 5872–5886. DOI: [10.1109/JSTARS.2020.3025790](https://doi.org/10.1109/JSTARS.2020.3025790).
- [10] L. Wang et al. "A Convolutional Neural Network-Based Method for Corn Stand Counting in the Field". In: *Sensors* 21.2 (2021), p. 507. DOI: [10.3390/s21020507](https://doi.org/10.3390/s21020507).
- [11] X. Luo et al. "Fast Automatic Vehicle Detection in UAV Images Using Convolutional Neural Networks". In: *Remote Sensing* 12.12 (2020), p. 1994. DOI: [10.3390/rs12121994](https://doi.org/10.3390/rs12121994).
- [12] N. B. Tatini et al. "Yolov4 Based Rice Fields Classification from High-Resolution Images Taken by Drones". In: *Proceedings of the IGARSS 2022-2022 IEEE International Geoscience and Remote Sensing Symposium*. IEEE. 2022, pp. 5043–5046.
- [13] R. Yang et al. "Lightweight Small Target Detection Algorithm with Multi-Feature Fusion". In: *Electronics* 12.12 (2023), p. 2739. DOI: [10.3390/electronics12122739](https://doi.org/10.3390/electronics12122739).
- [14] X. Luo, Y. Wu, and F. Wang. "Target Detection Method of UAV Aerial Imagery Based on Improved YOLOv5". In: *Remote Sensing* 14.19 (2022), p. 5063. DOI: [10.3390/rs14195063](https://doi.org/10.3390/rs14195063).

-
- [15] J.-F. Yeh et al. "Automatic Counting and Location Labeling of Rice Seedlings from Unmanned Aerial Vehicle Images". In: *Electronics* 13.1 (2024), p. 273. DOI: [10.3390/electronics13020273](https://doi.org/10.3390/electronics13020273).
 - [16] M. Yao et al. "Rice Counting and Localization in Unmanned Aerial Vehicle Imagery Using Enhanced Feature Fusion". In: *Agronomy* 14.1 (2024), p. 868. DOI: [10.3390/agronomy14040868](https://doi.org/10.3390/agronomy14040868).